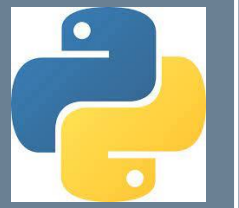


Afzal Hossain
Assistant Professor

Best Python Certifications

What is Python?

Python is a popular programming language. It was created by Guido van Rossum, and released in 1991.

I. It is used for:
II. Web development (server-side),
III. Software Development,
IV. Mathematics,
V. System Scripting.

What can Python do?

- Python can be used on a server to create web applications.
- Python can connect to database systems. It can also read and modify files.
- Python can be used to handle big data and perform complex mathematics.
- Python can be used for rapid prototyping, or for production-ready software development.

Why Python?

- Python works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc.).
- Python has a simple syntax similar to the English language.
- Python has syntax that allows developers to write programs with fewer lines than some other programming languages.
- Python runs on an interpreter system, meaning that code can be executed as soon as it is written.
- Python can be treated in a procedural way, an object-oriented way or a functional way.

Why Python?

- Data Analysis and Processing
- Artificial Intelligence
- Games
- Hardware/Sensor/Robots
- Desktop Applications

Python Features

1) Easy to Learn and Use
2) Expressive Language
3) Interpreted Language
4) Cross-platform Language
5) Free and Open Source
6) Object-Oriented Language
7) Extensible
8) Large Standard Library
9) GUI Programming Support
10) Integrated
11)Embeddable

Python History

- Python laid its foundation in the late 1980s. The implementation of Python was started in December 1989 by **Guido Van Rossum** at CWI in the Netherlands.
- In February 1991, **Guido Van Rossum** published the code (labeled version 0.9.0) to alt.sources. In 1994, Python 1.0 was released with new features like lambda, map, filter, and reduce.
- Python 2.0 added new features such as list comprehensions, and garbage collection systems.
- On December 3, 2008, Python 3.0 (also called "Py3K") was released. It was designed to rectify the fundamental flaw of the language.
- *ABC programming language* is said to be the predecessor of Python language, which was capable of Exception Handling and interfacing with the Amoeba Operating System. The following programming languages influence Python:
  - ABC language.
  - Modula-3

Why the Name Python?

- There is a fact behind choosing the name Python. **Guido van Rossum** was reading the script of a popular BBC comedy series "**Monty Python's Flying Circus**". It was late on-air 1970s.
- Van Rossum wanted to select a name which unique, sort, and little-bit mysterious. So he decided to select naming Python after **"Monty Python's Flying Circus"** for their newly created programming language.
- The comedy series was creative and well random. It talks about everything. Thus it is slow and unpredictable, which made it very interesting.

Python Applications

Web Applications: We can use Python to develop web applications. Python provides many useful frameworks, and these are given below:

- Django and Pyramid framework(Use for heavy applications)
- Flask and Bottle (Micro-framework)
- Plone and Django CMS (Advance Content management)

Python Applications

Desktop GUI Applications: The GUI stands for Graphical User Interface, which provides a smooth interaction to any application. Python provides a Tk GUI library to develop a user interface. Some popular GUI libraries are given below.

- Tkinter or Tk
- wxWidgetM
- Kivy (used for writing multitouch applications)
- PyQt or Pyside

Python Applications

Console-based Application: Console-based applications run from the command line or shell. These applications are computer program which are used commands to execute. This kind of application was more popular in the old generation of computers. Python can develop this kind of application very effectively. It is famous for having REPL, which means the Read-Eval-Print Loop makes it the most suitable language for command-line applications.

Python Applications

Software Development: Python is useful for the software development process. It works as a support language and can be used to build control and management, testing, etc.

- SCons is used to build control.
- Buildbot and Apache Gumps are used for automated continuous compilation and testing.
- Round or Trac for bug tracking and project management.

Python Applications

Scientific and Numeric: This is the era of Artificial intelligence where the machine can perform the task the same as the human. Python language is the most suitable language for Artificial intelligence or machine learning. It consists of many scientific and mathematical libraries, which makes easy to solve complex calculations. A few popular frameworks of machine libraries are given below.

- SciPy
- Scikit-learn
- NumPy
- Pandas
- Matplotlib

Python Applications

Business Applications: Business Applications differ from standard applications. E-commerce and ERP are an example of business applications. This kind of application requires scalability and readability, and Python provides all these features.

Oddo is an example of an all-in-one Python-based application that offers a range of business applications. Python provides a **Tryton** platform that is used to develop business applications.

Python Applications

Audio or Video-based Applications: Python is flexible to perform multiple tasks and can be used to create multimedia applications. Some multimedia applications which are made by using Python are Tim Player, cplay, etc. The few multimedia libraries are given below.

- Gstreamer
- Pyglet
- QT Phonon

Python Applications

3D CAD Applications: CAD (Computer Aided Design) is used to design engineering-related architecture. It is used to develop the 3D representation of a part of a system. Python can create a 3D CAD application by using the following functionalities.

- Fandango (Popular)
- CAMVOX
- HeeksCNC
- AnyCAD
- RCAM

Python Applications

Enterprise Applications: Python can be used to create applications that can be used within an Enterprise or an Organization. Some real-time applications are OpenERP, Tryton, Picalo, etc.

Image Processing Application: Python contains many libraries that are used to work with the image. The image can be manipulated according to our requirements. Some libraries of image processing are given below.

- OpenCV
- Pillow
- SimpleITK

Python Versions

- Python has two major versions: 2x and 3x.
- Python 2.x was released in 2000. The latest version is 2.7 released in 2010. It isn't recommended for use in new projects.
- Python 3.x was released in 2008. Basically, Python 3 isn't compatible with Python 2. And you should use the latest versions of Python 3 for your new projects.

Download and Install Python

You can download Python documentation from

https://www.python.org/downloads/

Check Your Python Version

› python --version

Run Your Python Program from

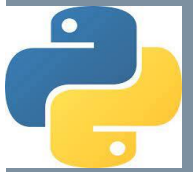
› python pname.py

› python PATH/pname.py

Change CMD to Python Terminal

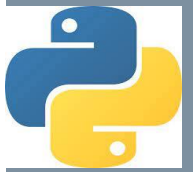
› Python

› exit()



Run your first Python Program

- Print (“Hello World”)
- Print (‘Hello World’)
  - Command Prompt
  - Python Terminal
  - Python GUI
  - Python File
  - Jupyter Notebook
  - Google Colab



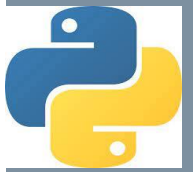
Python Indentation

Right Code-

```
if 5 > 2:
        print("Five is greater than two!")
```

Wrong Code-

```
if 5 > 2:
print("Five is greater than two!")
```



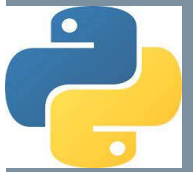
Python Indentation

Right Code-

```
if 5 > 2:
        print("Five is greater than two!")
if 5 > 2:
        print("Five is greater than two!")
```

Wrong Code-

```
if 5 > 2:
print("Five is greater than two!")
    print("Five is greater than two!")
```



Python Comments

- Comments can be used to explain Python code.
- Comments can be used to make the code more readable.
- Comments can be used to prevent execution when testing code.

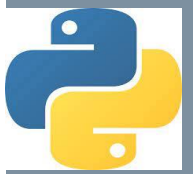
Creating a Comment: Comments start with a #, and Python will ignore them:

Single Line Comments :

```
#This is a comment
print("Hello, World!")
```

Multi-Line Comments :

```
"""
This is a comment written in
more than just one line
"""
print("Hello, World!")
```



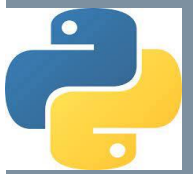
Python Variables

Variables: Variables are containers for storing data values.

Creating Variables: Python has no command for declaring a variable. A variable is created the moment you first assign a value to it.

```
x = 5
y = "John"
print(x)
print(y)
```

```
x = 4          # x is of type int
x = "Sally"    # x is now of type str
print(x)
```



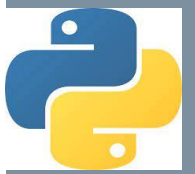
Python Variables

Casting: If you want to specify the data type of a variable, this can be done with casting.

```
x = str(3)    # x will be '3'
y = int(3)    # y will be 3
z = float(3)  # z will be 3.0
```

Get the Type: You can get the data type of a variable with the type() function.

```
x = 5
y = "John"
print(type(x))
print(type(y))
```



Python Variables

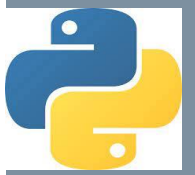
Single or Double Quotes?

String variables can be declared either by using single or double quotes:

```
x = "John"
# is the same as
x = 'John'
```

Case-Sensitive: Variable names are case-sensitive.

```
a = 4
A = "Sally"
#A will not overwrite a
```



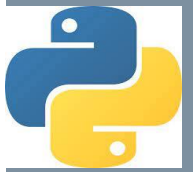
Python Variables

Naming Variables: When you name a variable, you need to adhere to some rules. If you don't, you'll get an error. The following are the variable rules that you should keep in mind:

- Variable names can contain only letters, numbers, and underscores (_). They can start with a letter or an underscore (_), not with a number.

- Variable names cannot contain spaces. To separate words in variables, you use underscores for example sorted_list.

- Variable names cannot be the same as keywords, reserved words, and built-in functions in Python.



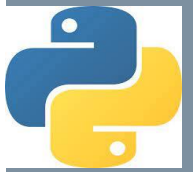
Python Variables

Naming Variables: The multi-word keywords can be created by the following method.

Camel Case - In the camel case, each word or abbreviation in the middle of begins with a capital letter. There is no intervention of whitespace. For example - nameOfStudent, valueOfVaraible, etc.

Pascal Case - It is the same as the Camel Case, but here the first word is also capitalized. For example - NameOfStudent, etc.

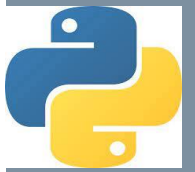
Snake Case - In the snake case, Words are separated by the underscore. For example - name_of_student, etc.



Python Variables

Naming variables: The following guidelines help you define good variable names:

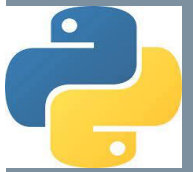
- Variable names should be concise and descriptive. For example, the active_user variable is more descriptive than the au.
- Use underscores (_) to separate multiple words in the variable names.
- Avoid using the letter l and the uppercase letter O because they look like the number 1 and 0.



Python Variables

Object Identity: In Python, every created object identifies uniquely in Python. Python provides the guarantee that no two objects will have the same identifier. The built-in **id()** function, is used to identify the object identifier. Consider the following example.

```
a = 50
b = a
print(id(a))
print(id(b))
# Reassigned variable a
a = 500
print(id(a))
```



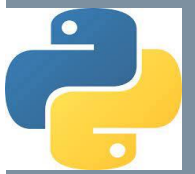
Python Variables

Multiple Assignment: Python allows us to assign a value to multiple variables in a single statement, which is also known as multiple assignments.

We can apply multiple assignments in two ways, either by assigning a single value to multiple variables or assigning multiple values to multiple variables. Consider the following example.

Assigning a single value to multiple variables

```
x=y=z=50
print(x)
print(y)
print(z)
```



Python Variables

Python Variable Types: There are two types of variables in Python - Local variable and Global variable. Let's understand the following variables.

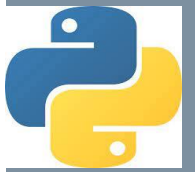
Local Variable: Local variables are the variables that are declared inside the function and have scope within the function. Let's understand the following example.

Right Code:

```
# Declaring a function
def add():
    # Defining local variables. They have scope only within a function
    a = 20
    b = 30
    c = a + b
    print("The sum is:", c)

# Calling a function
add()
```

Wrong Code:

```
add()
# Accessing local variable s outside the function
print(a)
```

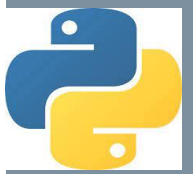


Python Variables

Python Variable Types:

Global Variables: Global variables can be used throughout the program, and their scope is in the entire program. We can use global variables inside or outside the function.

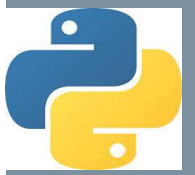
A variable declared outside the function is the global variable by default. Python provides the **global** keyword to use the global variable inside the function. If we don't use the **global** keyword, the function treats it as a local variable. Let's understand the following example.



Python Variables

Python Variable Types: Global Variables:

```
# Declare a variable and initialize it
x = 101

# Global variable in a function
def mainFunction():
    # printing a global variable
    global x
    print(x)
    # modifying a global variable
    x = 'Welcome To Javatpoint'
    print(x)

mainFunction()
print(x)
```



Python Variables

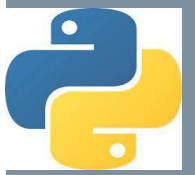
Delete a variable: We can delete the variable using the **del** keyword. The syntax is given below.

Syntax:

```
del <variable_name>
```

```
# Assigning a value to x
x = 6
print(x)
# deleting a variable.
del x
print(x)
```

```
# A Python program to display that we can store
 large numbers in Python
a = 10000000000000000000000000000000000000
00000000
a = a + 1
print(type(a))
print (a)
```



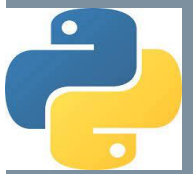
Python Variables

Example - 1 (Printing Single Variable)

```
# printing single value
a = 5
print(a)
print((a))
```

Example - 2 (Printing Multiple Variables)

```
a = 5
b = 6
# printing multiple variables
print(a,b)
# separate the variables by the comma
Print(1, 2, 3, 4, 5, 6, 7, 8)
```

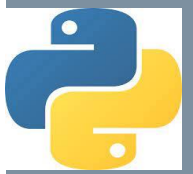


Python Variables

How to use string literals in Python?

```
strings = "This is Python"
char = "C"
multiline_str = """This is a multiline
string with more than one line code."""
unicode = u"\u00dcnic\u00f6de"
raw_str = r"raw \n string"

print(strings)
print(char)
print(multiline_str)
print(unicode) print(raw_str)
```

```
This is Python C This is a multiline
string with more than one line code.
Ünicöde
raw \n string
```



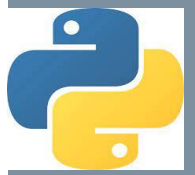
Python Variables

Boolean literals: A Boolean literal can have any of the two values: True or False.

Example 8: How to use boolean literals in Python?

```
x = (1 == True)
y = (1 == False)
a = True + 4
b = False + 10

print("x is", x)
print("y is", y)
print("a:", a)
print("b:", b)
```

```
x is True
 y is False
a: 5
b: 10
```

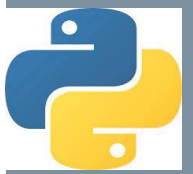


Python Variables

Example 9: How to use special literals in Python?

```
drink = "Available"
food = None

def menu(x):
    if x == drink:
        print(drink)
    else:
        print(food)

menu(drink)
menu(food)
```

```
Available
None
```

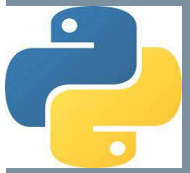


Python Variables

Example 10: How to use literals collections in Python?

```
fruits = ["apple", "mango", "orange"]
#list
numbers = (1, 2, 3) #tuple
alphabets = {'a':'apple', 'b':'ball',
'c':'cat'} #dictionary
vowels = {'a', 'e', 'i' , 'o', 'u'}
#set

print(fruits)
print(numbers)
print(alphabets)
print(vowels)
```

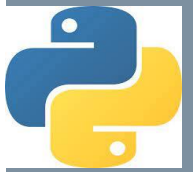
```
['apple', 'mango', 'orange']
(1, 2, 3)
{'a': 'apple', 'b': 'ball', 'c': 'cat'}
{'e', 'a', 'o', 'i', 'u'}
```



Python Data Types

Built-in Data Types: In programming, the data type is an important concept. Variables can store data of different types, and different types can do different things. Python has the following data types built-in by default, in these categories:

Text Type:	str
Numeric Types:	int, float, complex
Sequence Types:	list, tuple, range
Mapping Type:	dict
Set Types:	set, frozenset
Boolean Type:	bool
Binary Types:	bytes, bytearray, memoryview
None Type:	NoneType

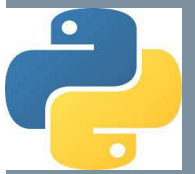


Python Data Types

Getting the Data Type: You can get the data type of any object by using the **type()** function:

Example: Print the data type of the variable x:

```
x = 5
print(type(x))
```



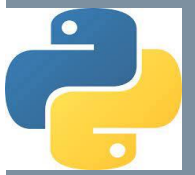
Python Data Types

Setting the Data Type: In Python, the data type is set when you assign a value to a variable:

```
Output:
Hello World
<class 'str'>
```

Example	Data Type	Code

x = "Hello World"	str	x = "Hello World"
		#display x:
		print(x)
		#display the data type of x:
		print(type(x))



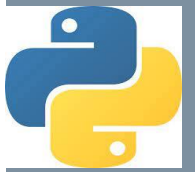
Python Data Types

Setting the Data Type: In Python, the data type is set when you assign a value to a variable:

```
Output
20
<class 'int'>
```

Example	Data Type	Code

x = 20	int	x = 20
		#display x:
		print(x)
		#display the data type of x:
		print(type(x))



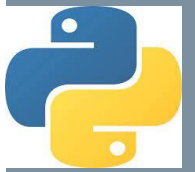
Python Data Types

Setting the Data Type: In Python, the data type is set when you assign a value to a variable:

```
Output:
20.5
<class 'float'>
```

Example	Data Type	Code

x = 20.5	float	x = 20.5
		#display x:
		print(x)
		#display the data type of x:
		print(type(x))



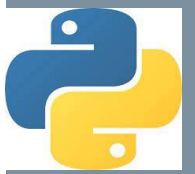
Python Data Types

Setting the Data Type: In Python, the data type is set when you assign a value to a variable:

```
Output:
1j
<class 'complex'>
```

Example	Data Type	Code

x = 1j	complex	x = 1j
		
		#display x:
		print(x)
		
		#display the data type of x:
		print(type(x))



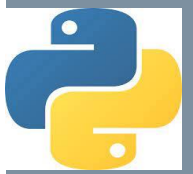
Python Data Types

Setting the Data Type: In Python, the data type is set when you assign a value to a variable:

```
Output:
['apple', 'banana', 'cherry']
<class 'list'>
```

Example	Data Type	Code

x = ["apple", "banana", "cherry"]	List
	
	Data from the List can be changed
		x = ["apple", "banana", "cherry"]
		
		#display x:
		print(x)
		
		#display the data type of x:
		print(type(x))



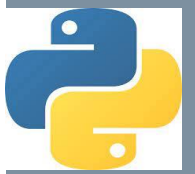
Python Data Types

Setting the Data Type: In Python, the data type is set when you assign a value to a variable:

Output:
('apple', 'banana', 'cherry')
<class 'tuple'>

Example	Data Type	Code

x = ("apple", "banana", "cherry")
Data from tuples can not be changed
	Tuple	x = ("apple", "banana", "cherry")
		
		#display x:
		print(x)
		
		#display the data type of x:
		print(type(x))



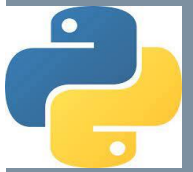
Python Data Types

Setting the Data Type: In Python, the data type is set when you assign a value to a variable:

```
Output:
range(0, 6)
<class 'range'>
```

Example	Data Type	Code

x = range(6)	range	x = range(6)
		
		#display x:
		print(x)
		
		#display the data type of x:
		print(type(x))



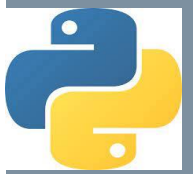
Python Data Types

Setting the Data Type: In Python, the data type is set when you assign a value to a variable:

```
Output:
{'name': 'John', 'age': 36}
<class 'dict'>
```

Example	Data Type	Code

x = {"name" : "John", "age" : 36}	dict	x = {"name" : "John", "age" : 36}
		
		#display x:
		print(x)
		
		#display the data type of x:
		print(type(x))



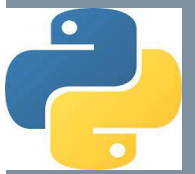
Python Data Types

Setting the Data Type: In Python, the data type is set when you assign a value to a variable:

```
Output:
{'cherry', 'apple', 'banana'}
<class 'set'>
```

Example	Data Type	Code

x = {"apple", "banana", "cherry"}	set	x = {"apple", "banana", "cherry"}
		
		#display x:
		print(x)
		
		#display the data type of x:
		print(type(x))



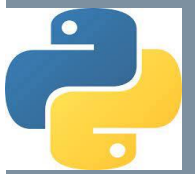
Python Data Types

Setting the Data Type: In Python, the data type is set when you assign a value to a variable:

```
Output:
frozenset({'apple', 'cherry',
'banana'})
<class 'frozenset'>
```

Example	Data Type	Code

x = frozenset({"apple", "banana", "cherry"})	frozenset	x = frozenset({"apple", "banana", "cherry"})
		
		#display x:
		print(x)
		
		#display the data type of x:
		print(type(x))



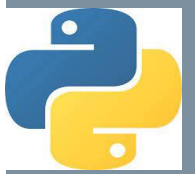
Python Data Types

Setting the Data Type: In Python, the data type is set when you assign a value to a variable:

```
Output:
True
<class 'bool'>
```

Example	Data Type	Code

x = True	bool	x = True
		
		#display x:
		print(x)
		
		#display the data type of x:
		print(type(x))



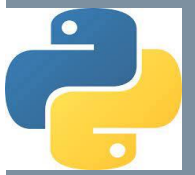
Python Data Types

Setting the Data Type: In Python, the data type is set when you assign a value to a variable:

```
Output:

x = b"Hello"
<class 'bytes'>
```

Example	Data Type	Code

x = b"Hello"	bytes	x = b"Hello"
		
		#display x:
		print(x)
		
		#display the data type of x:
		print(type(x))



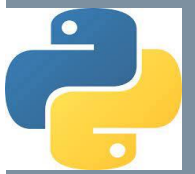
Python Data Types

Setting the Data Type: In Python, the data type is set when you assign a value to a variable:

```
Output:
bytearray(b'\x00\x00\x00\x00\x00')
<class 'bytearray'>
```

Example	Data Type	Code

x = bytearray(5)	bytearray	x = bytearray(5)
		
		#display x:
		print(x)
		
		#display the data type of x:
		print(type(x))



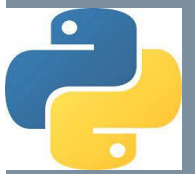
Python Data Types

Setting the Data Type: In Python, the data type is set when you assign a value to a variable:

```
Output:
<memory at 0x006F8FA0>
<class 'memoryview'>
```

Example	Data Type	Code

x = memoryview(bytes(5))	memoryview	x = memoryview(bytes(5))
		
		#display x:
		print(x)
		
		#display the data type of x:
		print(type(x))



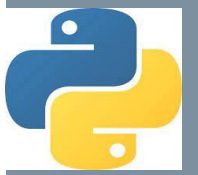
Python Data Types

Setting the Data Type: In Python, the data type is set when you assign a value to a variable:

```
Output:
None
<class 'NoneType'>
```

Example	Data Type	Code

x = None	NoneType	x = None
		
		#display x:
		print(x)
		
		#display the data type of x:
		print(type(x))

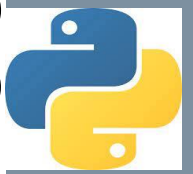


Python Data Types

Type Conversion: You can convert from one type to another with the int(), float(), and complex() methods:

```
x = 1    # int
y = 2.8  # float
z = 1j   # complex

#convert from int to float:
a = float(x)

#convert from float to int:
b = int(y)

#convert from int to complex:
c = complex(x)
```

```
print(a)
print(b)
print(c)

print(type(a))
print(type(b))
print(type(c))
```



Python Casting

Example
Integers:

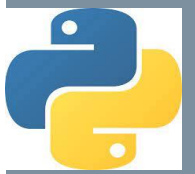
```
x = int(1)   # x will be 1
y = int(2.8) # y will be 2
z = int("3") # z will be 3
```

Example
Floats:

```
x = float(1)     # x will be 1.0
y = float(2.8)   # y will be 2.8
z = float("3")   # z will be 3.0
w = float("4.2") # w will be 4.2
```

Example
Strings:

```
x = str("s1") # x will be 's1'
y = str(2)    # y will be '2'
z = str(3.0)  # z will be '3.0'
```

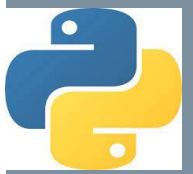


Python Booleans

Boolean Values: In programming, you often need to know if an expression is True or False. You can evaluate any expression in Python, and get one of two answers, True or False. When you compare two values, the expression is evaluated and Python returns the Boolean answer:

Example

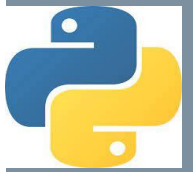
```
print(10 > 9)
print(10 == 9)
print(10 < 9)
```



Python Booleans

Example: Print a message based on whether the condition is True or False:

```
a = 200
b = 33

if b > a:
  print("b is greater than a")
else:
  print("b is not greater than a")
```



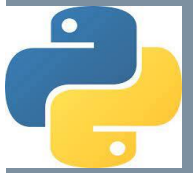
Python Random Number:

Example: Single Random

```
import random
x=random.randint(1,5)
print(x)

y=random.random()
print(y)
```

Example: Multiple random

```
m=[]
for i in range(0,5):
 n=random.randint(1,50)
 m.append(n)
 print(m)


y=random.sample(range(1,40),5)
print(y)
```

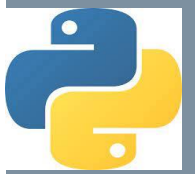


Python Operators

Python Operators: Operators are used to performing operations on variables and values. In the example below, we use the + operator to add together two values:

Example

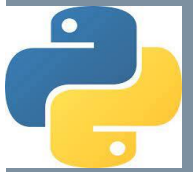
```
print(10 + 5)
```



Python Operators

Python divides the operators into the following groups:

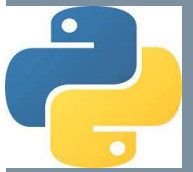
- Arithmetic operators
- Assignment operators
- Comparison operators
- Logical operators
- Identity operators
- Membership operators
- Bitwise operators



Python Operators

Arithmetic operators

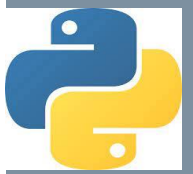
Operator	Name	Example
+	Addition	x + y
-	Subtraction	x - y
*	Multiplication	x * y
/	Division	x / y
%	Modulus	x % y
**	Exponentiation	x ** y
//	Floor division	x // y



Python Operators

Arithmetic operators

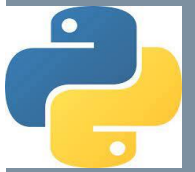
```
x = 5
y = 3

print(x + y)
print(x - y)
print(x * y)
print(x / y)
print(x % y)
print(x ** y)
print(x // y)
```



Python Operators

Assignment Operators

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
//=	x //= 3	x = x // 3
**=	x **= 3	x = x ** 3



Python Operators

Assignment Operators

```
x = 5
print(x)
```

```
x = 5
x += 3
print(x)
```

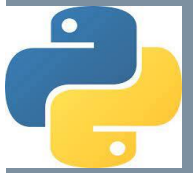
```
x = 5
x -= 3
print(x)
```

```
x = 5
x /= 3
print(x)
```

```
x = 5
x%=3
print(x)
```

```
x = 5
x//=3
print(x)
```

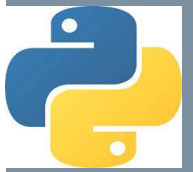
```
x = 5
x **= 3
print(x)
```



Python Operators

Bitwise Assignment Operators

Operator	Example	Same As
&=	x &= 3	x = x & 3
|=	x |= 3	x = x | 3
^=	x ^= 3	x = x ^ 3
>>=	x >>= 3	x = x >> 3
<<=	x <<= 3	x = x << 3



Python Operators

Assignment Operators

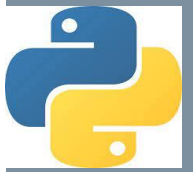
```
x = 5
x &= 3
print(x)
```

```
x = 5
x |= 3
print(x)
```

```
x = 5
x ^= 3
print(x)
```

```
x = 5
x >>= 3
print(x)
```

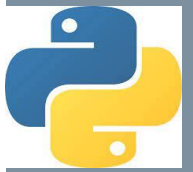
```
x = 5
x <<= 3
print(x)
```



Python Operators

❑ Comparison Operators

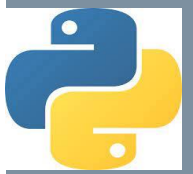
Operator	Name	Example
==	Equal	x == y
!=	Not equal	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y



Python Operators

- Comparison Operators

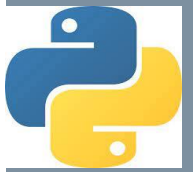
```
x = 5
y = 3
print(x == y)
# returns False because 5 is
not equal to 3

x = 5
y = 3
print(x != y)
# returns True because 5 is
not equal to 3
```



Python Operators

- Comparison Operators

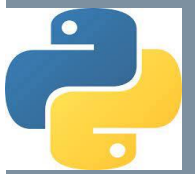
```
x = 5
y = 3
print(x > y)
# returns True because 5 is
greater than 3

x = 5
y = 3
print(x < y)
# returns False because 5 is
not less than 3
```



Python Operators

- Comparison Operators

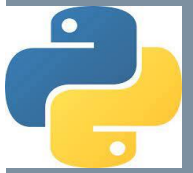
```
x = 5
y = 3
print(x >= y)
# returns True because five is greater,
or equal, to 3

x = 5
y = 3
print(x <= y)
# returns False because 5 is neither less
than or equal to 3
```



Python Operators

Logical Operators

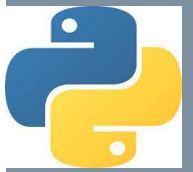
Operator	Description	Example
and	Returns True if both statements are true	x < 5 and x < 10
or	Returns True if one of the statements is true	x < 5 or x < 4
not	Reverse the result, and returns False if the result is true	not(x < 5 and x < 10)



Python Operators

Logical Operators

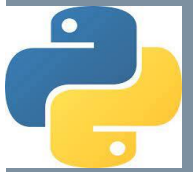
```
x = 5
print(x > 3 and x < 10)
# returns True because 5 is greater than 3 AND 5 is
less than 10

x = 5
print(x > 3 or x < 4)
# returns True because one of the conditions is
true (5 is greater than 3, but 5 is not less than
4)

x = 5
print(not(x > 3 and x < 10))
# returns False because not is used to reverse the
result
```



Python Operators

- Identity operators

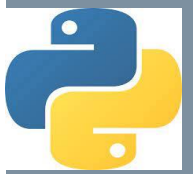
Operator	Description	Example
is	Returns True if both variables are the same object	x is y
is not	Returns True if both variables are not the same object	x is not y



Python Operators

- Identity operators

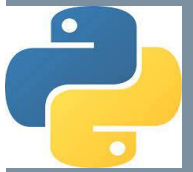
```
x = ["apple", "banana"]
y = ["apple", "banana"]
z = x
print(x is z)
# returns True because z is the same object as x

print(x is y)

# returns False because x is not the same object as
y, even if they have the same content

print(x == y)

# to demonstrate the difference betweeen "is" and
"==": this comparison returns True because x is
equal to y
```



Python Operators

❑ Identity operators

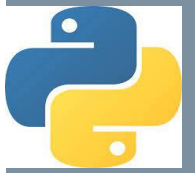
```
x = ["apple", "banana"]
y = ["apple", "banana"]
z = x
print(x is not z)
# returns False because z is the same object as x

print(x is not y)
# returns True because x is not the same object as y, even if
they have the same content

print(x != y)
# to demonstrate the difference between "is not" and "!=":
this comparison returns False because x is equal to y
```



Python Operators

- Membership operators

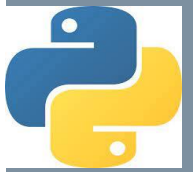
Operator	Description	Example
in	Returns True if a sequence with the specified value is present in the object	x in y
not in	Returns True if a sequence with the specified value is not present in the object	x not in y



Python Operators

- Membership operators

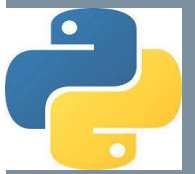
```
x = ["apple", "banana"]
print("banana" in x)
# returns True because a sequence with the
value "banana" is in the list

x = ["apple", "banana"]
print("pineapple" not in x)
# returns True because a sequence with the
value "pineapple" is not in the list
```



Python Operators

❑ Bitwise operators

Operator	Name	Description
&	AND	Sets each bit to 1 if both bits are 1
|	OR	Sets each bit to 1 if one of two bits is 1
^	XOR	Sets each bit to 1 if only one of two bits is 1
~	NOT	Inverts all the bits
<<	Zero fill left shift	Shift left by pushing zeros in from the right and let the leftmost bits fall off
>>	Signed right shift	Shift right by pushing copies of the leftmost bit in from the left, and let the rightmost bits fall off

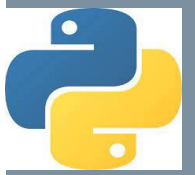


Python Strings

Strings: Strings in python are surrounded by either single quotation marks or double quotation marks.
'hello' is the same as "hello". You can display a string literal with the print() function:

Example

```
print("Hello")
print('Hello')
```

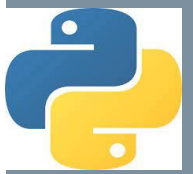


Python Strings

Assign String to a Variable: Assigning a string to a variable is done with the variable name followed by an equal sign and the string:

Example

```
a = "Hello"
print(a)
```

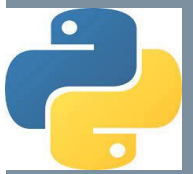


Python Strings

Multiline Strings: You can assign a multiline string to a variable by using three quotes:

Example

```
a = """Lorem ipsum dolor sit
amet,consectetur adipiscing
elit,sed do eiusmod tempor
incididuntut labore et dolore
magna aliqua."""
print(a)
```



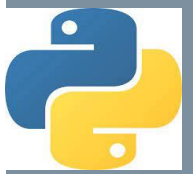
Python Strings

Strings are Arrays: Like many other popular programming languages, strings in Python are arrays of bytes representing Unicode characters.

However, Python does not have a character data type, a single character is simply a string with a length of 1.

Square brackets can be used to access elements of the string.

```
Example
a = "Hello, World!"
print(a[1])
```



Python Strings

Looping Through a String: Since strings are arrays, we can loop through the characters in a string, with a for loop.

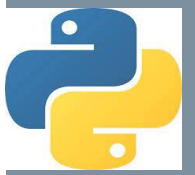
Example

```
for x in "banana":
  print(x)
```

String Length: To get the length of a string, use the len() function. The len() function returns the length of a string:

Example

```
a = "Hello, World!"
print(len(a))
```



Python Strings

Check String: To check if a certain phrase or character is present in a string, we can use the keyword in.

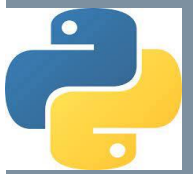
Example

```
txt = "The best things in life are free!"
print("free" in txt)
```

Use it in an if statement. Print only if "free" is present:

Example

```
txt = "The best things in life are!"
if "free" in txt:
  print("Yes, 'free' is present.")
else:
  print("Not Present")
```



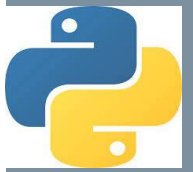
Python Strings

Slicing Strings: you can return a range of characters by using the slice syntax.
Specify the start index and the end index, separated by a colon, to return a part of the string.

```
Example
b = "Hello, World!"
print(b[2:5])
```

Slice From the Start: By leaving out the start index, the range will start at the first character:

```
Example
b = "Hello, World!"
print(b[:5])
```

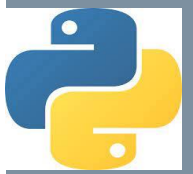


Python Strings

Negative Indexing: Use negative indexes to start the slice from the end of the string:

Example

```
b = "Hello, World!"
print(b[-5:-2])
```



Python Strings

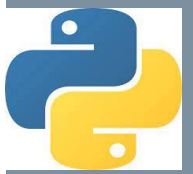
Modify Strings: Python has a set of built-in methods that you can use on strings.

Upper Case: The upper() method returns the string in upper case:

```
Example
a = "Hello, World!"
print(a.upper())
```

Lower Case: The lower() method returns the string in lower case:

```
Example
a = "Hello, World!"
print(a.lower())
```



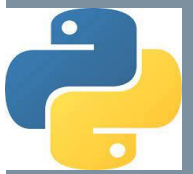
Python Strings

Modify Strings: Remove Whitespace: Whitespace is the space before and/or after the actual text, and very often you want to remove this space.

```
Example
a = "   Hello, World!    "
print(a.strip()) # returns "Hello, World!"
```

Replace String: The replace() method replaces a string with another string:

```
Example
a = "Hello, World!"
print(a.replace("H", "J"))
```



Python Strings

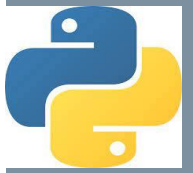
Modify Strings: Split String: The split() method returns a list where the text between the specified separator becomes the list items. The split() method splits the string into substrings if it finds instances of the separator:

```
Example
a = "Hello, World!"
print(a.split(","))
# returns ['Hello', ' World!']
```

String Concatenation: To concatenate, or combine, two strings you can use the + operator.

```
Example
a = "Hello"
b = "World"
c = a + b
print(c)
```

```
Example (Add “ ”)
a = "Hello"
b = "World"
c = a + " " + b
print(c)
```



Python Strings

String Format: As we learned in the Python Variables chapter, we cannot combine strings and numbers like this:

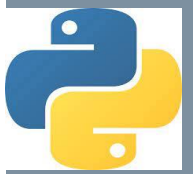
Example

```
age = 36
txt = "My name is John, I am " + age
print(txt)
```

But we can combine strings and numbers by using the format() method! The format() method takes the passed arguments, formats them, and places them in the string where the placeholders {} are:

Example: Use the format() method to insert numbers into strings:

```
age = 36
txt = "My name is John, and I am {}"
print(txt.format(age))
```

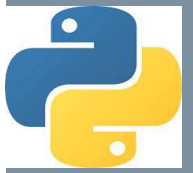


Python Strings

String Format: The format() method takes an unlimited number of arguments, and are placed into the respective placeholders:

Example

```
quantity = 3
itemno = 567
price = 49.95
myorder = "I want {} pieces of item {} for {}
Taka."
print(myorder.format(quantity, itemno, price))
```

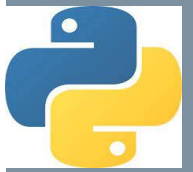


Python Strings

String Format: You can use index numbers {0} to be sure the arguments are placed in the correct placeholders:

Example

```
quantity = 3
itemno = 567
price = 49.95
myorder = "I want to pay {2} dollars for {0}
pieces of item {1}."
print(myorder.format(quantity, itemno, price))
```



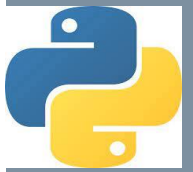
Python Strings

Escape Character: To insert characters that are illegal in a string, use an escape character. An escape character is a backslash \ followed by the character you want to insert. An example of an illegal character is a double quote inside a string that is surrounded by double quotes:

Example: You will get an error if you use double quotes inside a string that is surrounded by double quotes:

```
txt = "We are the so-called "Vikings" from the north."
```

#You will get an error if you use double quotes inside a string that are surrounded by double quotes:

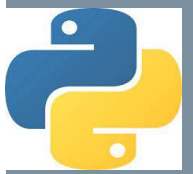


Python Strings

Escape Character: To fix this problem, use the escape character \":

Example: The escape character allows you to use double quotes when you normally would not be allowed:

```
txt = "We are the so-called \"Vikings\" from the north."
```

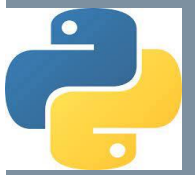


Python Strings

Escape Characters:

Code	Result
\'	Single Quote
\\	Backslash
\n	New Line
\r	Carriage Return

```
txt = 'It\'s alright.'
print(txt)

txt = "This will insert one \\
(backslash)."
print(txt)

txt = "Hello\nWorld!"
print(txt)

txt = "Hello\rWorld!"
print(txt)
```

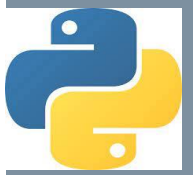


Python Strings

Escape Characters:

Code	Result
\t	Tab
\b	Backspace
\ooo	Octal value
\xhh	Hex value

```
txt = "Hello\tWorld!"
print(txt)

#This example erases one character
(backspace):
txt = "Hello \bWorld!"
print(txt)

#A backslash followed by three integers
will result in a octal value:
txt = "\110\145\154\154\157"
print(txt)

#A backslash followed by an 'x' and a hex
number represents a hex value:
txt = "\x48\x65\x6c\x6c\x6f"
print(txt)
```



Python Strings

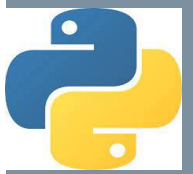
String Methods

Method	Description

capitalize()	Converts the first character to upper case
	Example :
	`txt = "hello, and welcome to my world."`
	`x = txt.capitalize()`
	`print (x)`

casefold()	Converts string into lower case
	Example:
	`txt = "Hello, And Welcome To My World!"`
	`x = txt.casefold()`
	`print(x)`

center()	Returns a centered string
	Example:
	`txt = "banana"`
	`x = txt.center(100)`
	`print(x)`



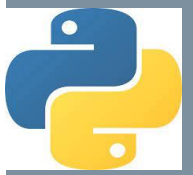
Python Strings

String Methods

Method	Description

count()	Returns the number of times a specified value occurs in a string
	Example: Return the number of times the value "apple" appears in the string:
	`txt = "I love apples, apple are my favorite fruit"`
	`x = txt.count("apple")`
	`print(x)`

encode()	Returns an encoded version of the string
	Example: UTF-8 encode the string:
	`txt = "My name is Ståle"`
	`x = txt.encode()`
	`print(x)`



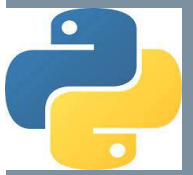
Python Strings

String Methods

Method	Description

endswith()	Returns true if the string ends with the specified value
	Example: Check if the string ends with a punctuation sign (.):
	`txt = "Hello, welcome to my world."`
	`x = txt.endswith(".")`
	`print(x)`

expandtabs()	Sets the tab size of the string
	Example: Set the tab size to 2 whitespaces:
	`txt = "H\te\tl\tl\to"`
	`x =  txt.expandtabs(2)`
	`print(x)`

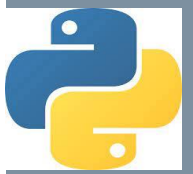


Python Strings

String Methods

Method	Description

find()	Searches the string for a specified value and returns the position of where it was found
	Example: Where in the text is the word "welcome"?:
	`txt = "Hello, welcome to my world."`
	`x = txt.find("welcome")`
	`print(x)`
	
	Where in the text is the first occurrence of the letter "e" when you only search between position 5 and 10?: if not found then return -1.
	`txt = "Hello, welcome to my world."`
	`x = txt.find("e", 5, 10)`
	`print(x)`



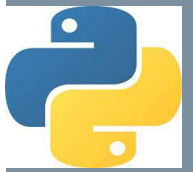
Python Strings

String Methods

Method	Description

format()	Formats specified values in a string
	Example: Insert the price inside the placeholder, the price should be in fixed point, two-decimal format:
	`txt = "For only {price:.2f} dollars!"`
	`print(txt.format(price = 49))`

index()	Searches the string for a specified value and returns the position of where it was found. Similar to find() method but if data not found index returns error but find return -1.

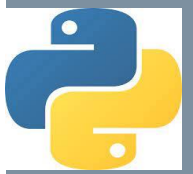


Python Strings

String Methods

Method	Description

isalnum()	Returns True if all characters in the string are alphanumeric. The isalnum() method returns True if all the characters are alphanumeric, meaning alphabet letter (a-z) and numbers (0-9). Example of characters that are not alphanumeric: (space)!#%&? etc.
	
	Example: Check if all the characters in the text are alphanumeric:
	`txt = "Company12"`
	`x = txt.isalnum()`
	`print(x)`
	
	`txt = "Company 12"`
	`x = txt.isalnum()`
	`print(x)`

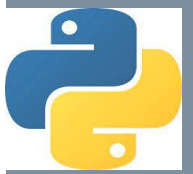


Python Strings

String Methods

Method	Description

isalpha()	Returns True if all characters in the string are in the alphabet.
	Example: Check if all the characters in the text are letters:
	
	`txt = "CompanyX"`
	`x = txt.isalpha()`
	`print(x)`
	
	`txt = "Company10"`
	`x = txt.isalpha()`
	`print(x)`

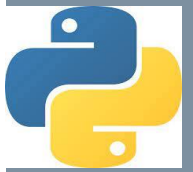


Python Strings

String Methods

Method	Description

isdecimal()	Returns True if all characters in the string are decimals
	Example: Check if all the characters in the Unicode object are decimals:
	
	`txt = "\u0033" #unicode for 3`
	`x = txt.isdecimal()`
	`print(x)`
	
	`txt = "33"`
	`x = txt.isdecimal()`
	`print(x)`

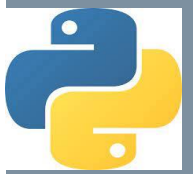


Python Strings

String Methods

Method	Description

isdigit()	Returns True if all characters in the string are digits
	Example: Check if all the characters in the text are digits:
	
	`txt = "50800"`
	`x = txt.isdigit()`
	`print(x)`

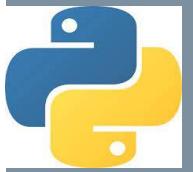


Python Strings

String Methods

Method	Description

isidentifier()	Returns True if the string is an identifier/ The isidentifier() method returns True if the string is a valid identifier, otherwise False. A string is considered a valid identifier if it only contains alphanumeric letters (a-z) and (0-9), or underscores (_). A valid identifier cannot start with a number, or contain any spaces. **Example:**
	`txt = "Demo"`
	`x = txt.isidentifier()`
	`print(x)`
	
	`txt = "Demo"`
	`x = txt.isidentifier()`
	`print(x)`

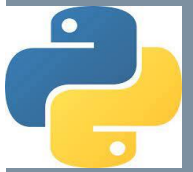


Python Strings

String Methods

Method	Description

islower()	Returns True if all characters in the string are lowercase.
	Example: Check if all the characters in the text are in lowercase:
	
	`txt = "hello world!"`
	`x = txt.islower()`
	`print(x)`

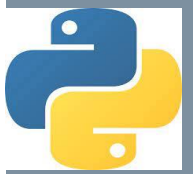


Python Strings

String Methods

Method	Description

isnumeric()	Returns True if all characters in the string are numeric. The isnumeric() method returns True if all the characters are numeric (0-9), otherwise False. Exponents, like ² and ¾ are also considered to be numeric values. "-1" and "1.5" are NOT considered numeric values, because all the characters in the string must be numeric, and the - and the . are not.
	Example: Check if all the characters in the text are numeric:
	
	`txt = "565543"`
	`x = txt.isnumeric()`
	`print(x)`

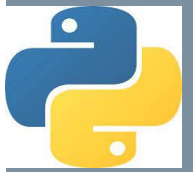


Python Strings

String Methods

Method	Description

isprintable()	Returns True if all characters in the string are printable
	Example: Check if all the characters in the text are printable:
	
	`txt = "Hello! Are you #1?"`
	`x = txt.isprintable()`
	`print(x)`
	
	`txt = "Hello!\nAre you #1?"`
	`x = txt.isprintable()`
	`print(x)`

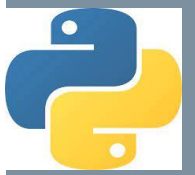


Python Strings

String Methods

Method	Description

isspace()	Returns True if all characters in the string are whitespaces
	Example: Check if all the characters in the text are whitespaces:
	
	`txt = "  "`
	`x = txt.isspace()`
	`print(x)`

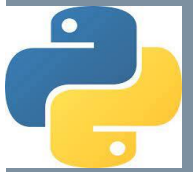


Python Strings

String Methods

Method	Description

istitle()	Returns True if the string follows the rules of a title
	Example: Check if each word start with an upper case letter:
	
	`txt = "Hello, And Welcome To My World!"`
	`x = txt.istitle()`
	`print(x)`

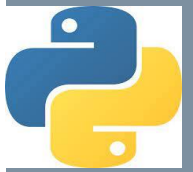


Python Strings

String Methods

Method	Description

isupper()	Returns True if all characters in the string are upper case
	Example: Check if all the characters in the text are in upper case:
	
	`txt = "THIS IS NOW!"`
	`x = txt.isupper()`
	`print(x)`

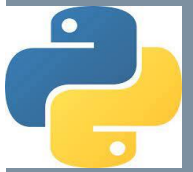


Python Strings

String Methods

Method	Description

join()	Joins the elements of an iterable to the end of the string
	Example: Join all items in a tuple into a string, using a hash character as separator:
	
	`myTuple = ("John", "Peter", "Vicky")`
	`x = "#".join(myTuple)`
	`print(x)`

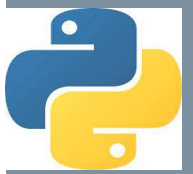


Python Strings

String Methods

Method	Description

ljust()
 	
rjust()
	Returns a left-justified version of the string.
	Example: Return 20 characters long, left-justified version of the word "banana":
	
	`txt = "banana"`
	`x = txt.ljust(20)`
	`print(x, "is my favorite fruit.")`
	
	Note: In the result, there are actually 14 whitespaces to the right of the word banana.
	`txt = "banana"`
	`x = txt.rjust(20)`
	`print(x, "is my favorite fruit.")`



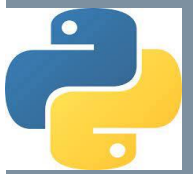
Python Strings

String Methods

Method	Description

lower()	Converts a string into lowercase
	Example: Lowercase the string:
	`txt = "Hello my FRIENDS"`
	`x = txt.lower()`
	`print(x)`

lstrip()	Returns a left trim version of the string
	Example: Remove spaces to the left of the string:
	`txt = "     banana     "`
	`print("of all fruits", txt, "is my favorite")`
	`x = txt.lstrip()`
	`print("of all fruits", x, "is my favorite")`

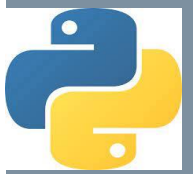


Python Strings

String Methods

Method	Description

rstrip()	Returns a left trim version of the string
	Example: Remove spaces to the left of the string:
	`txt = "     banana     "`
	`print("of all fruits", txt, "is my favorite")`
	`x = txt.rstrip()`
	`print("of all fruits", x, "is my favorite")`

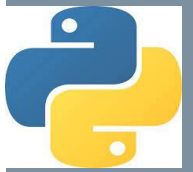


Python Strings

String Methods

Method	Description

maketrans()	Returns a translation table to be used in translations
	Example: Create a mapping table, and use it in the translate() method to replace any "S" characters with a "P" character:
	
	`txt = "Hello Sam!"`
	`mytable = txt.maketrans("S", "P")`
	`print(txt.translate(mytable))`

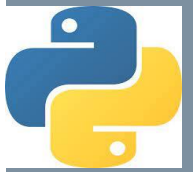


Python Strings

String Methods

Method	Description

partition()	Returns a tuple where the string is parted into three parts
	Example: Search for the word "bananas", and return a tuple with three elements:
	
	1 - everything before the "match"
	2 - the "match"
	3 - everything after the "match"
	
	`txt = "I could eat bananas all day"`
	`x = txt.partition("bananas")`
	`print(x)`



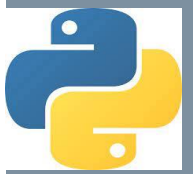
Python Strings

String Methods

Method	Description

replace()	Returns a string where a specified value is replaced with a specified value
	Example: Replace the word "bananas":
	`txt = "I like bananas"`
	`x = txt.replace("bananas", "apples")`
	`print(x)`

rfind()	Searches the string for a specified value and returns the last position of where it was found
	Example: Where in the text is the last occurrence of the string "casa"?:
	`txt = "Mi casa, su casa."`
	`x = txt.rfind("casa")`
	`print(x)`

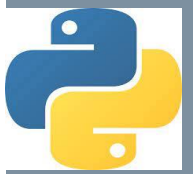


Python Strings

String Methods

Method	Description

rindex()	Searches the string for a specified value and returns the last position of where it was found
	Example: Where in the text is the last occurrence of the string "casa"?:
	
	`txt = "Mi casa, su casa."`
	`x = txt.rindex("casa")`
	`print(x)`



Python Strings

String Methods

Method	Description

rpartition()	Returns a tuple where the string is parted into three parts
	Example: Search for the last occurrence of the word "bananas", and return a tuple with three elements:
	1 - everything before the "match"
	2 - the "match"
	3 - everything after the "match"
	
	`txt = "I could eat bananas all day, bananas are my favorite fruit"`
	`x = txt.rpartition("bananas")`
	`print(x)`

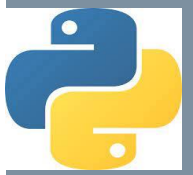
Python Strings

String Methods

Method	Description

rsplit()	Splits the string at the specified separator, and returns a list
	Example: Split a string into a list, using comma, followed by a space (,) as the separator:
	
	`txt = "apple, banana, cherry"`
	`x = txt.rsplit(", ")`
	`print(x)`

rstrip()	Returns a right trim version of the string
	Example: Remove any white spaces at the end of the string:
	
	`txt = "     banana     "`
	`x = txt.rstrip()`
	`print("of all fruits", x, "is my favorite")`



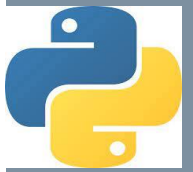
Python Strings

String Methods

Method	Description

split()	Splits the string at the specified separator, and returns a list
	Example: Split a string into a list where each word is a list item:
	
	`txt = "welcome to the jungle"`
	`x = txt.split()`
	`print(x)`

splitlines()	Splits the string at line breaks and returns a list
	Example: Split a string into a list where each line is a list item:
	
	`txt = "Thank you for the music\nWelcome to the jungle"`
	`x = txt.splitlines()`
	`print(x)`



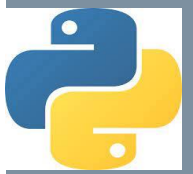
Python Strings

String Methods

Method	Description

startswith()	Returns true if the string starts with the specified value
	Example: Check if the string starts with "Hello":
	
	`txt = "Hello, welcome to my world."`
	`x = txt.startswith("Hello")`
	`print(x)`

strip()	Returns a trimmed version of the string
	Example: Remove spaces at the beginning and at the end of the string:
	
	`txt = "     banana     "`
	`x = txt.strip()`
	`print("of all fruits", x, "is my favorite")`



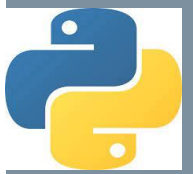
Python Strings

String Methods

Method	Description

swapcase()	Swaps cases, lower case becomes upper case and vice versa
	Example: Make the lower case letters upper case and the upper case letters lower case:
	
	`txt = "Hello My Name Is PETER"`
	`x = txt.swapcase()`
	`print(x)`

title()	Converts the first character of each word to upper case
	Example: Make the first letter in each word upper case:
	
	`txt = "Welcome to my world"`
	`x = txt.title()`
	`print(x)`



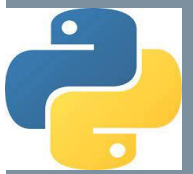
Python Strings

String Methods

Method	Description

translate()	Returns a translated string
	Example: Replace any "S" characters with a "P" character:
	
	`#use a dictionary with ASCII codes to replace 83 (S) with 80 (P):`
	`mydict = {83:  80}`
	`txt = "Hello Sam!"`
	`print(txt.translate(mydict))`

upper()	Converts a string into upper case
	Example-Upper case the string:
	
	`txt = "Hello my friends"`
	`x = txt.upper()`
	`print(x)`

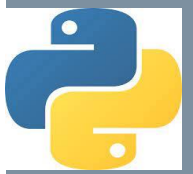


Python Strings

String Methods

Method	Description

zfill()	Fills the string with a specified number of 0 values at the beginning
	Example: Fill the string with zeros until it is 10 characters long:
	
	`txt = "50"`
	`x = txt.zfill(10)`
	`print(x)`

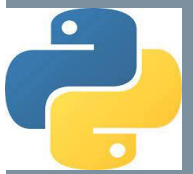


Python List

List: Lists are used to store multiple items in a single variable. Lists are one of 4 built-in data types in Python used to store collections of data, the other 3 are Tuple, Set, and Dictionary, all with different qualities and usage. Lists are created using square brackets:

Example of Creating a List:

```
thislist = ["apple", "banana", "cherry"]
print(thislist)
```

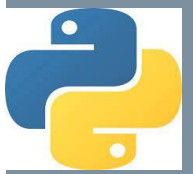


Python List

List Items: List items are ordered, changeable, and allow duplicate values. List items are indexed, the first item has index [0], the second item has index [1] etc.

Ordered: When we say that lists are ordered, it means that the items have a defined order, and that order will not change.
If you add new items to a list, the new items will be placed at the end of the list.

Changeable: The list is changeable, meaning that we can change, add, and remove items in a list after it has been created.



Python List

Allow Duplicates: Since lists are indexed, lists can have items with the same value:

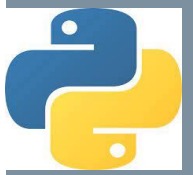
Example: Lists allow duplicate values:

```
thislist=["apple","banana", "cherry", "apple", "cherry"]
print(thislist)
```

List Length: To determine how many items a list has, use the len() function:

Example: Print the number of items in the list:

```
thislist = ["apple", "banana", "cherry"]
print(len(thislist))
```



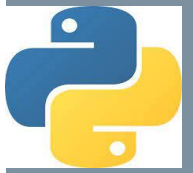
Python List

List Items - Data Types: List items can be of any data type. For Example String, int, and Boolean data types:

```
list1 = ["apple", "banana", "cherry"]
list2 = [1, 5, 7, 9, 3]
list3 = [True, False, False]
print(list1)
print(list2)
print(list3)
```

A list can contain different data types. For example A list with strings, integers, and Boolean values:

```
list1 = ["abc", 34, True, 40, "male"]
print(list1)
```



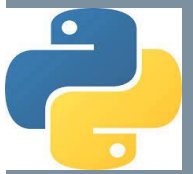
Python List

type(): From Python's perspective, lists are defined as objects with the data type 'list': <class 'list'>
Example: What is the data type of a list?

```
mylist = ["apple", "banana", "cherry"]
print(type(mylist))
```

The list() Constructor: It is also possible to use the list() constructor when creating a new list. Example: Using the list() constructor to make a List:

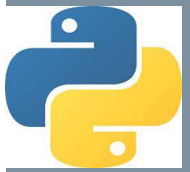
```
thislist = list(("apple", "banana", "cherry"))
# note the double round-brackets
print(thislist)
```



Python List

Python Collections (Arrays): There are four collection data types in the Python programming language:

List is a collection that is ordered and changeable. Allows duplicate members.
Tuple is a collection that is ordered and unchangeable. Allows duplicate members.
Set is a collection that is unordered, unchangeable*, and unindexed. No duplicate members. Set items are unchangeable, but you can remove and/or add items whenever you like.
Dictionary is a collection that is ordered** and changeable. No duplicate members. As of Python version 3.7, dictionaries are ordered. In Python 3.6 and earlier, dictionaries are unordered.



Python List

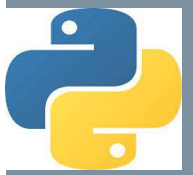
Access List Items: List items are indexed and you can access them by referring to the index number:
Example: Print the second item of the list:

```
thislist = ["apple", "banana", "cherry"]
print(thislist[1])
```

Note: The first item has index 0.

Negative Indexing: Negative indexing means start from the end. -1 refers to the last item, -2 refers to the second last item etc.
Example: Print the last item of the list:

```
thislist = ["apple", "banana", "cherry"]
print(thislist[-1])
```



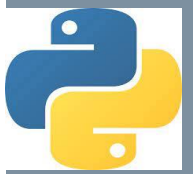
Python List

Range of Indexes: You can specify a range of indexes by specifying where to start and where to end the range. When specifying a range, the return value will be a new list with the specified items.

Example: Return the third, fourth, and fifth item:

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]
print(thislist[2:5])
```

Note: The search will start at index 2 (included) and end at index 5 (not included).



Python List

Remember that the first item has an index 0. By leaving out the start value, the range will start at the first item:

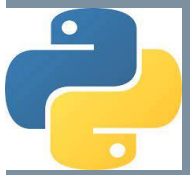
Example: This example returns the items from the beginning to, but NOT including, "kiwi":

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]
print(thislist[:4])
```

By leaving out the end value, the range will go on to the end of the list:

Example: This example returns the items from "cherry" to the end:

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]
print(thislist[2:])
```



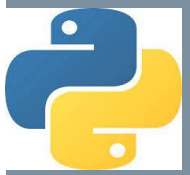
Python List

Range of Negative Indexes: Specify negative indexes if you want to start the search from the end of the list:
Example: This example returns the items from "orange" (-4) to, but NOT including "mango" (-1):

```
thislist = ["apple", "banana", "cherry", "orange",
"kiwi", "melon", "mango"]
print(thislist[-4:-1])
```

Check if Item Exists: To determine if a specified item is present in a list use the in a keyword: **Example:** Check if "apple" is present in the list:

```
thislist = ["apple", "banana", "cherry"]
if "apple" in thislist:
  print("Yes, 'apple' is in the fruits list")
```



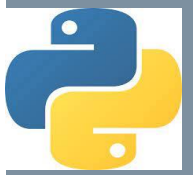
Python List

Change List Items: To change the value of a specific item, refer to the index number:

```
thislist = ["apple", "banana", "cherry"]
thislist[1] = "blackcurrant"
print(thislist)
```

Change a Range of Item Values:

```
thislist =
["apple", "banana", "cherry", "orange", "kiw
i", "mango"]
thislist[1:3] =
["blackcurrant", "watermelon"]
print(thislist)
```



Python List

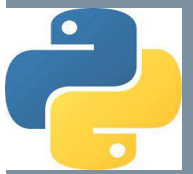
Change List Items: If you insert *more* items than you replace, the new items will be inserted where you specified, and the remaining items will move accordingly:

```
thislist =
["apple", "banana", "cherry"]
thislist[1:2] =
["blackcurrant", "watermelon"]
print(thislist)
```

Insert Items: To insert a new list item, without replacing any of the existing values, we can use the insert() method.

The insert() method inserts an item at the specified index:

```
thislist =
["apple", "banana", "cherry"]
thislist.insert(2, "watermelon")
print(thislist)
```



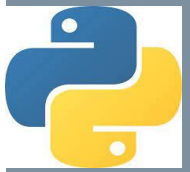
Python List

Change List Items: Append Items: To add an item to the end of the list, use the append() method:

```
thislist =
["apple", "banana", "cherry"]
thislist.append("orange")
print(thislist)
```

Extend List: To append elements from another list to the current list, use the extend() method.

```
thislist =
["apple", "banana", "cherry"]
tropical =
["mango", "pineapple", "papaya"]
thislist.extend(tropical)
print(thislist)
```



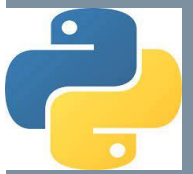
Python List

Remove Specified Item: The remove() method removes the specified item.

```
thislist = ["apple", "banana", "cherry"]
thislist.remove("banana")
print(thislist)
```

Remove Specified Index: The pop() method removes the specified index.

```
thislist = ["apple", "banana", "cherry"]
thislist.pop(1)
print(thislist)
```



Python List (Remove Item)

Remove Specified Item: The remove() method removes the specified item.

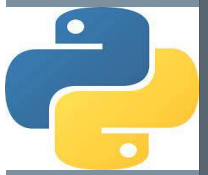
```
thislist = ["apple", "banana", "cherry"]
thislist.remove("banana")
print(thislist)this list
```

Remove Specified Index: The pop() method removes the specified index.

```
thislist = ["apple", "banana", "cherry"]
thislist.pop(1)
print(thislist)
```

If you do not specify the index, the pop() method removes the last item.

```
thislist.pop()
```



Python List (Remove Item)

Remove Specified Item: The del keyword also removes the specified index:

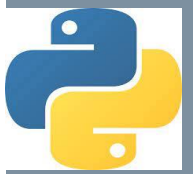
```
thislist = ["apple", "banana", "cherry"]
del thislist[0]
print(thislist)
```

Remove Specified Index: The del keyword can also delete the list completely.

```
thislist = ["apple", "banana", "cherry"]
del thislist
```

Clear the List: The clear() method empties the list. The list still remains, but it has no content.

```
thislist = ["apple", "banana", "cherry"]
thislist.clear()
print(thislist)
```



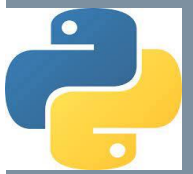
Python List (Loop Lists)

Loop Through a List: You can loop through the list items by using a for loop:

```
thislist = ["apple", "banana", "cherry"]
for x in thislist:
  print(x)
```

Loop Through the Index Numbers: It can also loop through the list of items by referring to their index number. Use the range() and len() functions to create a suitable iterable.

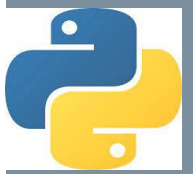
```
thislist = ["apple", "banana", "cherry"]
for i in range(len(thislist)):
  print(thislist[i])
```



Python List (Loop Lists)

Using a While Loop: You can loop through the list of items by using a while loop. Use the len() function to determine the length of the list, then start at 0 and loop your way through the list items by referring to their indexes. Remember to increase the index by 1 after each iteration.

```
thislist = ["apple", "banana", "cherry"]
i = 0
while i < len(thislist):
  print(thislist[i])
  i = i + 1
```



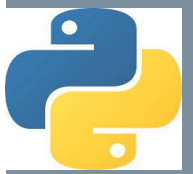
Python List (List Comprehension)

List Comprehension : List comprehension offers a shorter syntax when you want to create a new list based on the values of an existing list.

Example: Based on a list of fruits, you want a new list, containing only the fruits with the letter "a" in the name.

Without list comprehension you will have to write a for statement with a conditional test inside:

```
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
newlist = []

for x in fruits:
  if "a" in x:
    newlist.append(x)

print(newlist)
```



Python List (List Comprehension)

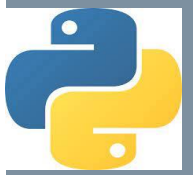
List Comprehension using Condition :

```
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
newlist = [x for x in fruits if x != "apple"]
print(newlist)
```

```
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
newlist = [x for x in fruits]
print(newlist)
```

```
newlist = [x for x in range(10)]
print(newlist)
```

```
newlist = [x for x in range(10) if x < 5]
print(newlist)
```



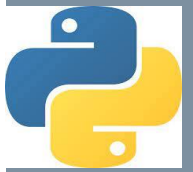
Python List (List Comprehension)

List Comprehension using Condition :

```
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
newlist = [x.upper() for x in fruits]
print(newlist)
```

```
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
newlist = ['hello' for x in fruits]
print(newlist))
```

```
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
newlist = [x if x != "banana" else "orange" for x in
fruits]
print(newlist)
```

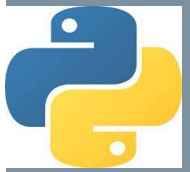


Python List (Sort Lists)

Sort List Alphanumerically: List objects have a sort() method that will sort the list alphanumerically, ascending, by default:

```
thislist = ["orange", "mango", "kiwi", "pineapple",
"banana"]
thislist.sort()
print(thislist)
```

```
thislist = [100, 50, 65, 82, 23]
thislist.sort()
print(thislist)
```

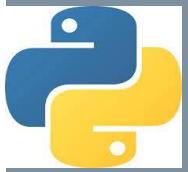


Python List (Sort Lists)

Sort Descending: To sort descending, use the keyword argument reverse = True: :

```
thislist = ["orange", "mango", "kiwi", "pineapple",
"banana"]
thislist.sort(reverse = True)
print(thislist)
```

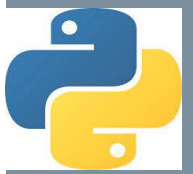
```
thislist = [100, 50, 65, 82, 23]
thislist.sort(reverse = True)
print(thislist)
```



Python List (Sort Lists)

Reverse Order: What if you want to reverse the order of a list, regardless of the alphabet? The reverse() method reverses the current sorting order of the elements.

```
thislist = ["banana", "Orange", "Kiwi", "cherry"]
thislist.reverse()
print(thislist)
```



Python List (Copy Lists)

Copy a List: You cannot copy a list simply by typing list2 = list1, because: list2 will only be a reference to list1, and changes made in list1 will automatically also be made in list2. There are ways to make a copy, one way is to use the built-in List method copy().

```
thislist = ["apple", "banana", "cherry"]
mylist = thislist.copy()
print(mylist)

Or

thislist = ["apple", "banana", "cherry"]
mylist = list(thislist)
print(mylist)
```

Python List (Join Lists)

Join Two Lists: There are several ways to join, or concatenate, two or more lists in Python. One of the easiest ways are by using the + operator.

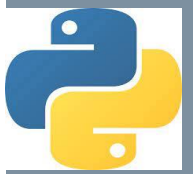
```
list1 = ["a", "b", "c"]
list2 = [1, 2, 3]
list3 = list1 + list2
print(list3)
```

Or

```
list1 = ["a", "b" , "c"]
list2 = [1, 2, 3]
for x in list2:
  list1.append(x)
print(list1)
```

Or

```
list1 = ["a", "b" , "c"]
list2 = [1, 2, 3]

list1.extend(list2)
print(list1)
```



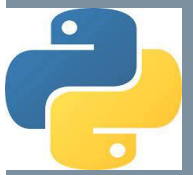
Python List (List Method)

List Methods: Python has a set of built-in methods that you can use on lists.

Method	Description

append()	Adds an element at the end of the list
	`list1 = ["a", "b" , "c"]`
	`list2 = [1, 2, 3]`
	`for x in list2:`
	`  list1.append(x)`
	`print(list1)`

clear()	Removes all the elements from the list
	`fruits = ["apple", "banana", "cherry"]`
	`fruits.clear()`
	`print(fruits)`



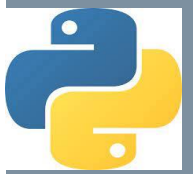
Python List (List Method)

List Methods: Python has a set of built-in methods that you can use on lists.

Method	Description

copy()	Returns a copy of the list
	`fruits = ["apple", "banana", "cherry"]`
	`x = fruits.copy()`
	`print(x)`

count()	Returns the number of elements with the specified value
	`fruits = ["apple", "banana", "cherry"]`
	`x = fruits.count("cherry")`
	`print(x)`



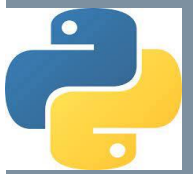
Python List (List Method)

List Methods: Python has a set of built-in methods that you can use on lists.

Method	Description

extend()	Add the elements of a list (or any iterable), to the end of the current list
	`fruits = ['apple', 'banana', 'cherry']`
	`cars = ['Ford', 'BMW', 'Volvo']`
	`fruits.extend(cars)`
	`print(fruits)`

index()	Returns the index of the first element with the specified value
	`fruits = ['apple', 'banana', 'cherry']`
	`x = fruits.index("cherry")`
	`print(x)`



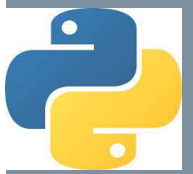
Python List (List Method)

List Methods: Python has a set of built-in methods that you can use on lists.

Method	Description

insert()	Adds an element at the specified position
	`fruits = ['apple', 'banana', 'cherry']`
	`fruits.insert(1, "orange")`
	`print(fruits)`

pop()	Removes the element at the specified position
	`fruits = ['apple', 'banana', 'cherry']`
	`fruits.pop(1)`
	`print(fruits)`



Python List (List Method)

List Methods: Python has a set of built-in methods that you can use on lists.

Method	Description

remove()	Removes the item with the specified value
	`fruits = ['apple', 'banana', 'cherry']`
	`fruits.remove("banana")`
	`print(fruits)`

reverse()	Reverses the order of the list
	`fruits = ['apple', 'banana', 'cherry']`
	`fruits.reverse()`
	`print(fruits)`

sort()	Sorts the list
	`cars = ['Ford', 'BMW', 'Volvo']`
	`cars.sort()`
	`print(cars)`

Python Tuples (Basic)

Tuple Items: Tuple items are ordered, unchangeable, and allow duplicate values.

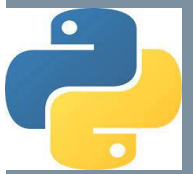
Tuple items are indexed, the first item has index [0], the second item has index [1] etc.

Ordered: When we say that tuples are ordered, it means that the items have a defined order, and that order will not change.

Unchangeable: Tuples are unchangeable, meaning that we cannot change, add or remove items after the tuple has been created.

Allow Duplicates: Since tuples are indexed, they can have items with the same value:

```
Thistuple=("apple", "banana", "cherry", "apple",
"cherry")
print(thistuple)
```



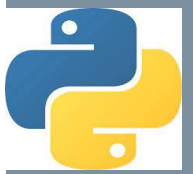
Python Tuples (Basic)

Tuple Length: To determine how many items a tuple has, use the len() function:

```
thistuple = ("apple", "banana", "cherry")
print(len(thistuple))
```

Create Tuple With One Item: To create a tuple with only one item, you have to add a comma after the item, otherwise Python will not recognize it as a tuple.

```
thistuple = ("apple",)
print(type(thistuple))
#NOT a tuple
thistuple = ("apple")
print(type(thistuple))
```



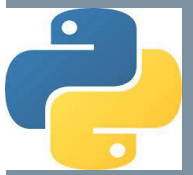
Python Tuples (Basic)

Tuple Items - Data Types: Tuple items can be of any data type

```
tuple1 = ("apple", "banana", "cherry")
tuple2 = (1, 5, 7, 9, 3)
tuple3 = (True, False, False)
print(tuple1)
print(tuple2)
print(tuple3)
```

A tuple can contain different data types

```
tuple1 = ("abc", 34, True, 40, "male")
print(tuple1)
```



Python Tuples (Basic)

type(): From Python's perspective, tuples are defined as objects with the data type 'tuple':

```
mytuple = ("apple", "banana", "cherry")
print(type(mytuple))
```

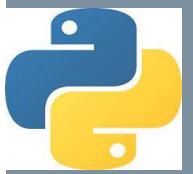
The tuple() Constructor: It is also possible to use the tuple() constructor to make a tuple.

```
thistuple = tuple(("apple", "banana", "cherry"))
# note the double round-brackets
print(thistuple)
```

Python Tuples (Basic)

Python Collections (Arrays): There are four collection data types in the Python programming language:

- List is a collection which is ordered and changeable. Allows duplicate members.
- Tuple is a collection which is ordered and unchangeable. Allows duplicate members.
- Set is a collection which is unordered, unchangeable*, and unindexed. No duplicate members.
- Dictionary is a collection which is ordered** and changeable. No duplicate members.



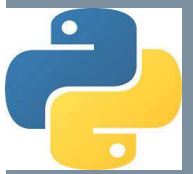
Python Tuples (Access Tuple Items)

Access Tuple Items: You can access tuple items by referring to the index number, inside square brackets:

```
thistuple = ("apple", "banana", "cherry")
print(thistuple[1])
Or
thistuple = ("apple", "banana", "cherry")
print(thistuple[-1])
```

Range of Indexes: You can specify a range of indexes by specifying where to start and where to end the range.When specifying a range, the return value will be a new tuple with the specified items.

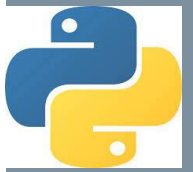
```
thistuple =
("apple", "banana", "cherry", "orange", "kiwi", "melon", "man
go")
print(thistuple[2:5])
```



Python Tuples (Access Tuple Items)

Access Tuple Items:

```
Thistuple=("apple", "banana", "cherry", "orange"
, "kiwi", "melon", "mango")
print(thistuple[:4])

Thistuple=("apple", "banana", "cherry", "orange"
, "kiwi", "melon", "mango")
print(thistuple[2:])

Thistuple=("apple", "banana", "cherry", "orange"
, "kiwi", "melon", "mango")
print(thistuple[-4:-1])
```



Python Tuples (Access Tuple Items)

Check if Item Exists: To determine if a specified item is present in a tuple use the in keyword:

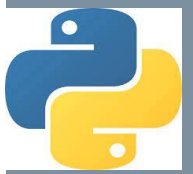
```
thistuple = ("apple", "banana", "cherry")
if "apple" in thistuple:
  print("Yes, 'apple' is in the fruits tuple")
```

Python Tuples (Update Tuple)

Change Tuple Values: Once a tuple is created, you cannot change its values. Tuples are unchangeable, or immutable as it also is called.

But there is a workaround. You can convert the tuple into a list, change the list, and convert the list back into a tuple.

```
x = ("apple", "banana", "cherry")
y = list(x)
y[1] = "kiwi"
x = tuple(y)
print(x)
```

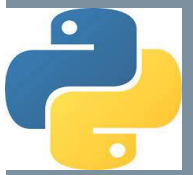


Python Tuples (Update Tuple)

Add Items: Since tuples are immutable, they do not have a build-in append() method, but there are other ways to add items to a tuple.

1. Convert into a list: Just like the workaround for changing a tuple, you can convert it into a list, add your item(s), and convert it back into a tuple.

```
thistuple = ("apple", "banana", "cherry")
y = list(thistuple)
y.append("orange")
thistuple = tuple(y)

print(thistuple)
```

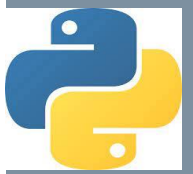


Python Tuples (Update Tuple)

Remove Items: Tuples are unchangeable, so you cannot remove items from it, but you can use the same workaround as we used for changing and adding tuple items:

Convert the tuple into a list, remove "apple", and convert it back into a tuple:

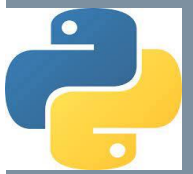
```
thistuple = ("apple", "banana", "cherry")
y = list(thistuple)
y.remove("apple")
thistuple = tuple(y)

print(thistuple)
```



Python Tuples (Update Tuple)

Remove Items: The del keyword can delete the tuple completely:

```
thistuple = ("apple", "banana", "cherry")
del thistuple
print(thistuple) #this will raise an error
because the tuple no longer exists
```



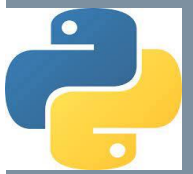
Python Tuples (Unpack Tuples)

Unpacking a Tuple: When we create a tuple, we normally assign values to it. This is called "packing" a tuple:

```
fruits = ("apple", "banana", "cherry")
print(fruits)
```

But, in Python, we are also allowed to extract the values back into variables. This is called "unpacking":

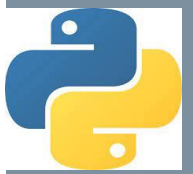
```
fruits = ("apple", "banana", "cherry")
(green, yellow, red) = fruits
print(green)
print(yellow)
print(red)
```



Python Tuples (Unpack Tuples)

Using Asterisk*: If the number of variables is less than the number of values, you can add an * to the variable name and the values will be assigned to the variable as a list:

```
fruits = ("apple", "banana", "cherry", "strawberry",
 "raspberry")

(green, yellow, *red) = fruits

print(green)
print(yellow)
print(red)
```



Python Tuples (Loop Tuples)

Loop Through a Tuple: You can loop through the tuple items by using a for loop.

```
thistuple = ("apple", "banana", "cherry")
for x in thistuple:
  print(x)
```

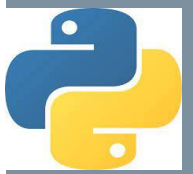
Loop Through the Index Numbers: You can also loop through the tuple items by referring to their index number. Use the range() and len() functions to create a suitable iterable.

```
thistuple = ("apple", "banana", "cherry")
for i in range(len(thistuple)):
  print(thistuple[i])
```

Python Tuples (Loop Tuples)

Using a While Loop: You can loop through the tuple items by using a while loop. Use the len() function to determine the length of the tuple, then start at 0 and loop your way through the tuple items by referring to their indexes. Remember to increase the index by 1 after each iteration.

```
thistuple = ("apple", "banana", "cherry")
i = 0
while i < len(thistuple):
  print(thistuple[i])
  i = i + 1
```



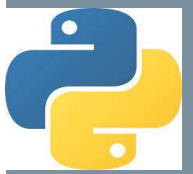
Python Tuples (Join Tuples)

Join Two Tuples: To join two or more tuples you can use the + operator:

```
tuple1 = ("a", "b" , "c")
tuple2 = (1, 2, 3)
tuple3 = tuple1 + tuple2
print(tuple3)
```

Multiply Tuples: If you want to multiply the content of a tuple a given number of times, you can use the * operator:

```
fruits = ("apple", "banana", "cherry")
mytuple = fruits * 2
print(mytuple)
```



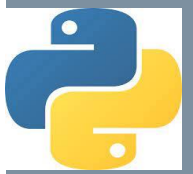
Python Tuples (Tuples Methods)

Tuple Methods: Python has two built-in methods that you can use on tuples.

Method	Description
count()	Returns the number of times a specified value occurs in a tuple
index()	Searches the tuple for a specified value and returns the position of where it was found

```
thistuple = (1, 3, 7, 8, 7, 5, 4, 6, 8, 5)
x = thistuple.count(5)
print(x)
```

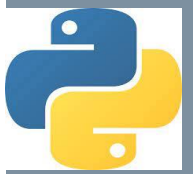
```
thistuple = (1, 3, 7, 8, 7, 5, 4, 6, 8, 5)
x = thistuple.index(8)
print(x)
```



Python Set (Definition)

Set: Sets are used to store multiple items in a single variable. A set is a collection which is unordered, unchangeable*, and unindexed.

```
x = {1, 3, 7, 8, 7, 5, 4}
print(x)
```



Python Set (Definition)

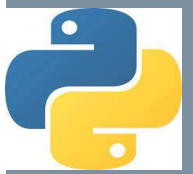
Set Items: Set items are unordered, unchangeable, and do not allow duplicate values.

Unordered: Unordered means that the items in a set do not have a defined order. Set items can appear in a different order every time you use them, and cannot be referred to by index or key.

Unchangeable: Set items are unchangeable, meaning that we cannot change the items after the set has been created. Once a set is created, you cannot change its items, but you can remove items and add new items.

Duplicates Not Allowed: Sets cannot have two items with the same value. If given, it will remove duplicate automatically.

```
x = {1, 3, 7, 8, 7, 5, 4}
print(x)
```

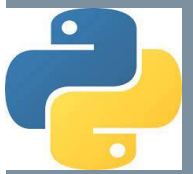


Python Set (Access Set Items)

Access Items: You cannot access items in a set by referring to an index or a key. But you can loop through the set items using a for loop or ask if a specified value is present in a set, by using the in keyword.

```
thisset = {"apple", "banana", "cherry"}
for x in thisset:
  print(x)

print("banana" in thisset)
```

Change Items: Once a set is created, you cannot change its items, but you can add new items.



Python Set (Add Set Items)

Add Items: Once a set is created, you cannot change its items, but you can add new items. To add one item to a set use the add() method.

```
thisset = {"apple", "banana", "cherry"}
thisset.add("orange")
print(thisset)
```

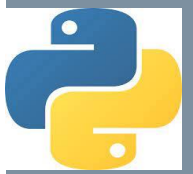
Add Sets: To add items from another set into the current set, use the update() method.

```
thisset = {"apple", "banana", "cherry"}
tropical = {"pineapple", "mango", "papaya"}
thisset.update(tropical)
print(thisset)
```

Python Set (Add Set Items)

Add Any Iterable: The object in the update() method does not have to be a set, it can be any iterable object (tuples, lists, dictionaries etc.).

```
thisset = {"apple", "banana", "cherry"}
mylist = ["kiwi", "orange"]
thisset.update(mylist)
print(thisset)
```



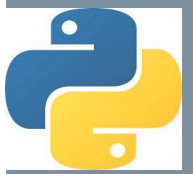
Python Set (Remove Set Items)

Remove Item: To remove an item in a set, use the remove(), the discard(), or pop() method.

```
thisset = {"apple", "banana", "cherry"}
thisset.remove("banana")
print(thisset)
```

```
thisset = {"apple", "banana", "cherry"}
thisset.discard("banana")
print(thisset)
```

```
thisset = {"apple", "banana", "cherry"}
x = thisset.pop()
print(x) #removed item
print(thisset) #the set after removal
```



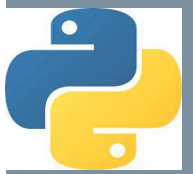
Python Set (Remove Set Items)

Remove Item: To remove an item in a set, use the remove(), the discard(), or pop() method.

```
thisset = {"apple", "banana", "cherry"}
thisset.remove("banana")
print(thisset)
```

```
thisset = {"apple", "banana", "cherry"}
thisset.discard("banana")
print(thisset)
```

```
thisset = {"apple", "banana", "cherry"}
x = thisset.pop()
print(x) #removed item
print(thisset) #the set after removal
```



Python Set (Remove Set Items)

The clear() and del method empties the set:

```
thisset = {"apple", "banana", "cherry"}
thisset.clear()
print(thisset)
```

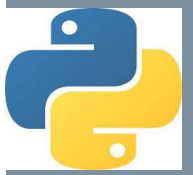
```
thisset = {"apple", "banana", "cherry"}
del thisset
print(thisset)
```

Python Set (Join Sets)

Join Two Sets: There are several ways to join two or more sets in Python. You can use the union() method that returns a new set containing all items from both sets, or the update() method that inserts all the items from one set into another:

```
set1 = {"a", "b" , "c"}
set2 = {1, 2, 3}
set3 = set1.union(set2)
print(set3)
```

```
set1 = {"a", "b" , "c"}
set2 = {1, 2, 3}
set1.update(set2)
print(set1)
```



Note: Both union() and update() will exclude any duplicate items.

Python Set (Join Sets)

Keep ONLY the Duplicates: The intersection_update() method will keep only the items that are present in both sets.

```
x = {"apple", "banana", "cherry"}
y = {"google", "microsoft", "apple"}
x.intersection_update(y)
print(x)
```

The intersection() method will return a *new* set, that

only contains the items that are present in both sets.

Return a set that contains the items that exist in both set x, and set y:

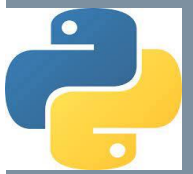
```
x = {"apple", "banana", "cherry"}
y = {"google", "microsoft", "apple"}
z = x.intersection(y)
print(z)
```

Python Set (Join Sets)

Keep All, But NOT the Duplicates:
The symmetric_difference_update() meth
od will keep only the elements that are
NOT present in both sets.

```
x = {"apple", "banana", "cherry"}
y = {"google", "microsoft", "apple"}
x.symmetric_difference_update(y)
print(x)
```

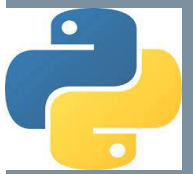
```
x = {"apple", "banana", "cherry"}
y = {"google", "microsoft", "apple"}
z = x.symmetric_difference(y)
print(z)
```



Python Set (Join Sets)

Note: The values True and 1 are considered the same value in sets, and are treated as duplicates:

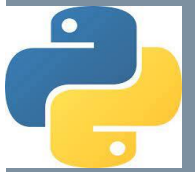
```
x = {"apple", "banana", "cherry", True}
y = {"google", 1, "apple", 2}
z = x.symmetric_difference(y)
```



Python Set (Set Methods)

Set Methods: Python has a set of built-in methods that you can use on sets.

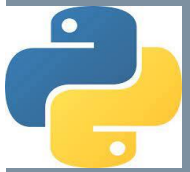
Method	Description
add()	Adds an element to the set
clear()	Removes all the elements from the set
copy()	Returns a copy of the set
difference()	Returns a set containing the difference between two or more sets
difference_update()	Removes the items in this set that are also included in another, specified set
discard()	Remove the specified item
intersection()	Returns a set, that is the intersection of two other sets
intersection_update()	Removes the items in this set that are not present in other, specified set(s)



Python Set (Set Methods)

Set Methods: Python has a set of built-in methods that you can use on sets.

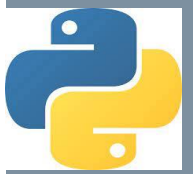
Method	Description
isdisjoint()	Returns whether two sets have a intersection or not
issubset()	Returns whether another set contains this set or not
issuperset()	Returns whether this set contains another set or not
pop()	Removes an element from the set
remove()	Removes the specified element
symmetric_difference()	Returns a set with the symmetric differences of two sets
symmetric_difference_update()	inserts the symmetric differences from this set and another
union()	Return a set containing the union of sets
update()	Update the set with the union of this set and others



Python Dictionary (Basic)

Dictionary: Dictionaries are used to store data values in key: value pairs. A dictionary is a collection which is ordered*, changeable and do not allow duplicates. As of Python version 3.7, dictionaries are *ordered*. In Python 3.6 and earlier, dictionaries are *unordered*. Dictionaries are written with curly brackets, and have keys and values:

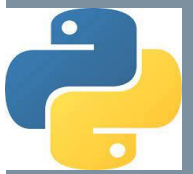
```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
print(thisdict)
```



Python Dictionary (Basic)

Dictionary Items: Dictionary items are ordered, changeable, and does not allow duplicates.
Dictionary items are presented in key: value pairs, and can be referred to by using the key name.

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
print(thisdict["brand"])
```



Python Dictionary (Basic)

Ordered or Unordered?

As of Python version 3.7, dictionaries are ordered. In Python 3.6 and earlier, dictionaries are unordered.

When we say that dictionaries are ordered, it means that the items have a defined order, and that order will not change.

Unordered means that the items does not have a defined order, you cannot refer to an item by using an index.

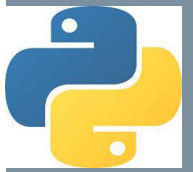
Changeable

Dictionaries are changeable, meaning that we can change, add or remove items after the dictionary has been created.

Duplicates Not Allowed

Dictionaries cannot have two items with the same key:

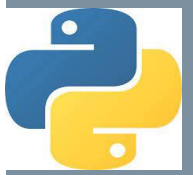
```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964,
  "year": 2020
}
print(thisdict)
```



Python Dictionary (Basic)

Dictionary Length: To determine how many items a dictionary has, use the len() function:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964,
  "year": 2020
}
print(thisdict)
print(len(thisdict))
```



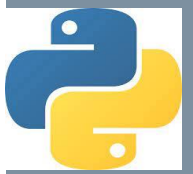
Python Dictionary (Access Dictionary Items)

Accessing Items: You can access the items of a dictionary by referring to its key name, inside square brackets:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
x = thisdict["model"]
```

There is also a method called get() that will give you the same result:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
x = thisdict.get("model")
print(x)
```



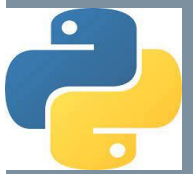
Python Dictionary(Access Dictionary Items)

Get Keys: The keys() method will return a list of all the keys in the dictionary.

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
x = thisdict.keys()
print(x)
```

Add a new item to the original dictionary, and see that the keys list gets updated as well:

```
car = {
"brand": "Ford",
"model": "Mustang",
"year": 1964
}
x = car.keys()
print(x) #before the change
car["color"] = "white"
print(x) #after the change
```



Python Dictionary (Access Dictionary Items)

Get Values: The values() method will return a list of all the values in the dictionary.

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
x = thisdict.values()

print(x)
```

Make a change in the original dictionary, and see that the values list gets updated as well:

```
car = {
"brand": "Ford",
"model": "Mustang",
"year": 1964
}
x = car.values()
print(x) #before the change
car["year"] = 2020
print(x) #after the change
```

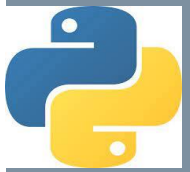
Python Dictionary(Access Dictionary Items)

Add a new item to the original dictionary, and see that the values list gets updated as well:

```
car = {
"brand": "Ford",
"model": "Mustang",
"year": 1964
}
x = car.values()
print(x) #before the change
car["color"] = "red"
print(x) #after the change
```

Get Items: The items() method will return each item in a dictionary, as tuples in a list.

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
x = thisdict.items()
print(x)
```



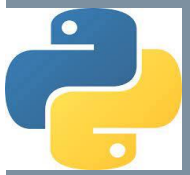
Python Dictionary (Access Dictionary Items)

Make a change in the original dictionary, and see that the items list gets updated as well:

```
car = {
"brand": "Ford",
"model": "Mustang",
"year": 1964
}
x = car.items()
print(x) #before the change
car["year"] = 2020
print(x) #after the change
```

Add a new item to the original dictionary, and see that the items list gets updated as well:

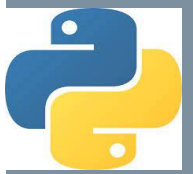
```
car = {
"brand": "Ford",
"model": "Mustang",
"year": 1964
}
x = car.items()
print(x) #before the change
car["color"] = "red"
print(x) #after the change
```



Python Dictionary(Access Dictionary Items)

Check if Key Exists: To determine if a specified key is present in a dictionary use the in keyword:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
if "model" in thisdict:
  print("Yes, 'model' is one of the keys in the thisdict dictionary")
```



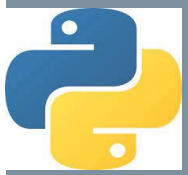
Python Dictionary (Change Dictionary Items)

Change Values: You can change the value of a specific item by referring to its key name:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}

thisdict["year"] = 2018

print(thisdict)
```

Update Dictionary: The update() method will update the dictionary with the items from the given argument. The argument must be a dictionary, or an iterable object with key:value pairs.

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
thisdict.update({"year": 2020})
```



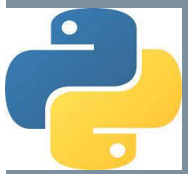
Python Dictionary (Add Dictionary Items)

Adding Items: Adding an item to the dictionary is done by using a new index key and assigning a value to it:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
thisdict["color"] = "red"
print(thisdict)
```

Update Dictionary: The update() method will update the dictionary with the items from the given argument. The argument must be a dictionary, or an iterable object with key:value pairs.

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
thisdict.update({"year": 2020})
```



Python Dictionary (Remove Dictionary Items)

Removing Items: There are several methods to remove items from a dictionary: The pop() method removes the item with the specified key name:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
thisdict.pop("model")
print(thisdict)
```

The popitem() method removes the last inserted item (in versions before 3.7, a random item is removed instead):

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
thisdict.popitem()
print(thisdict)
```

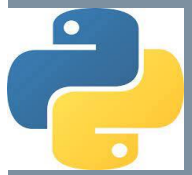
Python Dictionary (Remove Dictionary Items)

The del keyword removes the item with the specified key name:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
del thisdict["model"]
print(thisdict)
```

The del keyword can also delete the dictionary completely:

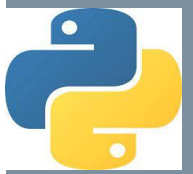
```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
del thisdict
print(thisdict)
#this will cause an error
because "thisdict" no longer
exists.
```



Python Dictionary (Remove Dictionary Items)

The clear() method empties the dictionary:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
thisdict.clear()
print(thisdict)
```



Python Dictionary(Loop Dictionaries)

Loop Through a Dictionary: You can loop through a dictionary by using a for loop. When looping through a dictionary, the return value are the *keys* of the dictionary, but there are methods to return the *values* as well. Print all key names in the dictionary, one by one:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
for x in thisdict:
  print(x)
```

Print all *values* in the dictionary, one by one:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
for x in thisdict:
  print(thisdict[x])
```

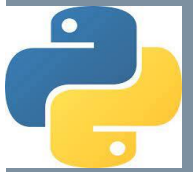
Python Dictionary(Loop Dictionaries)

You can also use the values() method to return values of a dictionary:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
for x in
thisdict.values():
  print(x)
```

You can use the keys() method to return the keys of a dictionary:

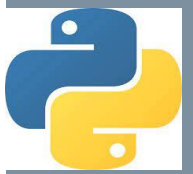
```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
for x in thisdict.keys():
  print(x)
```



Python Dictionary (Loop Dictionaries)

Loop through both *keys* and *values*, by using the items() method:

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
for x, y in
thisdict.items():
  print(x, y)
```



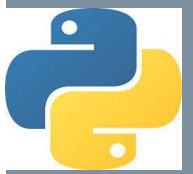
Python Dictionary (Copy Dictionaries)

Copy a Dictionary: You cannot copy a dictionary simply by typing dict2 = dict1, because: dict2 will only be a *reference* to dict1, and changes made in dict1 will automatically also be made in dict2. There are ways to make a copy, one way is to use the built-in Dictionary method copy().

```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
mydict =
thisdict.copy()
print(mydict)
```

Make a copy of a dictionary with the dict() function:

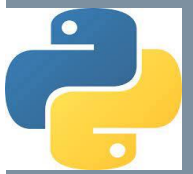
```
thisdict = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
mydict = dict(thisdict)
print(mydict)
```



Python Dictionary(Nested Dictionaries)

Nested Dictionaries: A dictionary can contain dictionaries, this is called nested dictionaries. Create a dictionary that contain three dictionaries:

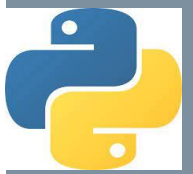
```
myfamily = {
  "child1" : {
    "name" : "Emil",
    "year" : 2004
  },
  "child2" : {
    "name" : "Tobias",
    "year" : 2007
  },
  "child3" : {
    "name" : "Linus",
    "year" : 2011
  }
}
```



Python Dictionary (Nested Dictionaries)

Create three dictionaries, then create one dictionary that will contain the other three dictionaries:

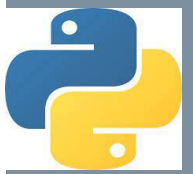
```
child1 = {
  "name" : "Emil",
  "year" : 2004
}
child2 = {
  "name" : "Tobias",
  "year" : 2007
}
child3 = {
  "name" : "Linus",
  "year" : 2011
}

myfamily = {
  "child1" : child1,
  "child2" : child2,
  "child3" : child3
}
```



Python Dictionary(Nested Dictionaries)

Access Items in Nested Dictionaries: To access items from a nested dictionary, you use the name of the dictionaries, starting with the outer dictionary:

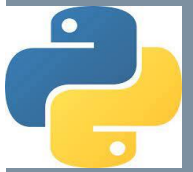
```
myfamily = {
  "child1" : {
    "name" : "Emil",
    "year" : 2004
  },
  "child2" : {
    "name" : "Tobias",
    "year" : 2007
  },
  "child3" : {
    "name" : "Linus",
    "year" : 2011
  }
}

print(myfamily["child2"]["name"])
```



Python Dictionary(Dictionary Methods)

Dictionary Methods: Python has a set of built-in methods that you can use on dictionaries.

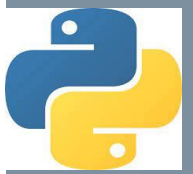
Method	Description
clear()	Removes all the elements from the dictionary
copy()	Returns a copy of the dictionary
fromkeys()	Returns a dictionary with the specified keys and value
get()	Returns the value of the specified key
items()	Returns a list containing a tuple for each key value pair
keys()	Returns a list containing the dictionary's keys
pop()	Removes the element with the specified key



Python Dictionary (Dictionary Methods)

Dictionary Methods: Python has a set of built-in methods that you can use on dictionaries.

Method	Description
popitem()	Removes the last inserted key-value pair
setdefault()	Returns the value of the specified key. If the key does not exist: insert the key, with the specified value
update()	Updates the dictionary with the specified key-value pairs
values()	Returns a list of all the values in the dictionary



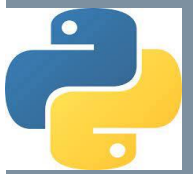
Python (Input and Output)

Input and Output Functions: A program needs to interact with the user to accomplish the desired task; this can be achieved using Input-Output functions. The **input()** function helps to enter data at run time by the user and the output function **print()** is used to display the result of the program on the screen after execution.

Output Functions: **The print() function**: In Python, the **print()** function is used to display result on the screen. The syntax for print() is as follows:

Example

```
print (“string to be displayed as output ” )
```

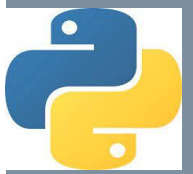


Python (Input and Output)

Input Functions: In Python, **input( )** function is used to accept data as input at run time. The syntax for **input()** function is,

Input Single Variable:

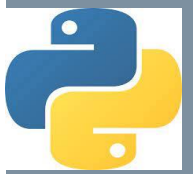
```
Variable = input (“prompt string”)
print(Variable)
OR
x = int (input(“Enter Number 1: ”))
y = int (input(“Enter Number 2: ”))
print (“The sum = ”, x+y)
OR
x = float (input("Enter Number 1: "))
y = float (input("Enter Number 2: "))
print ("The sum = ", x+y)
```



Python (Input and Output)

Input Functions: Input List Variable:

```
# creating an empty list
x = []

# number of elements as input
n = int(input("Enter number of elements : "))

# iterating till the range
for i in range(0, n):
    y = int(input())

    x.append(y) # adding the element

print(x)
```



Python (Input and Output)

Input Functions: Input Tuple Variable:

```
# creating an empty list
x = []

# number of elements as input
n = int(input("Enter number of elements : "))

# iterating till the range
for i in range(0, n):
    y = int(input())
    x.append(y) # adding the element

print(tuple(x))
```

Python (Input and Output)

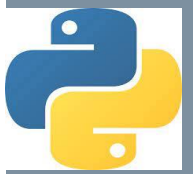
Input Functions: Input Set Variable:

```
# creating an empty list
x = set()

# number of elements as input
n = int(input("Enter number of elements : "))

# iterating till the range
for i in range(0, n):
    y = int(input())

    x.add(y) # adding the element
print(x)
```

Python (Input and Output)

Input Functions: Input Dictionary Variable:

```
# creating an empty list
x = {}

# number of elements as input
n = int(input("Enter number of elements : "))

for i in range(0,n):
    key = input("Enter employee's name: ")
    value = input("Enter employee's salary: ")

    x[key] = value
print(x)
```



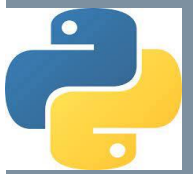
Python Conditions

Python Conditions and If statements: Python supports the usual logical conditions from mathematics:

I. Equals: a == b
II. Not Equals: a != b
III. Less than: a < b
IV. Less than or equal to: a <= b
V. Greater than: a > b
VI. Greater than or equal to: a >= b

These conditions can be used in several ways, most commonly in "if statements" and loops. An "if statement" is written by using the if keyword.

```
a = 33
b = 200
if b > a:
  print("b is greater than a")
```



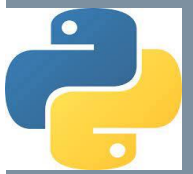
Python Conditions

If statement, without indentation (will raise an error):

```
a = 33
b = 200
if b > a:
print("b is greater than a")
# you will get an error
```

Elif: The elif keyword is Python's way of saying "if the previous conditions were not true, then try this condition".

```
a = 33
b = 33
if b > a:
  print("b is greater than a")
elif a == b:
  print("a and b are equal")
```



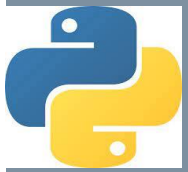
Python Conditions

Else: The else keyword catches anything which isn't caught by the preceding conditions.

```
a = 200
b = 33
if b > a:
  print("b is greater than a")
elif a == b:
  print("a and b are equal")
else:
  print("a is greater than b")
```

You can also have an else without the elif:

```
a = 200
b = 33
if b > a:
  print("b is greater than a")
else:
  print("b is not greater than a")
```



Python Conditions

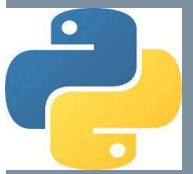
Shorthand If: If you have only one statement to execute, you can put it on the same line as the if statement. One line if statement:

```
a = 200
b = 33
if a > b: print("a is greater
than b")
```

One line if else statement:

```
a = 2
b = 330
print("A") if a > b else print("B")
```

This technique is known as **Ternary Operators**, or **Conditional Expressions**.



Python Conditions

Shorthand If: One line if else statement, with 3 conditions:

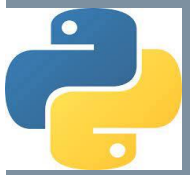
```
a = 330
b = 330
print("A") if a > b else print("=") if a == b else print("B")
```

And: The and keyword is a logical operator, and is used to combine conditional statements: Test if a is greater than b, AND if c is greater than a:

```
a = 200
b = 33
c = 500
if a > b and c > a:
  print("Both conditions are True")
```

Or: The or keyword is a logical operator, and is used to combine conditional statements:

```
a = 200
b = 33
c = 500
if a > b or a > c:
  print("At least one of the
conditions is True")
```



Python Conditions

Not: The not keyword is a logical operator, and is used to reverse the result of the conditional statement:

```
a = 33
b = 200
if not a > b:
  print("a is NOT greater than b")
```

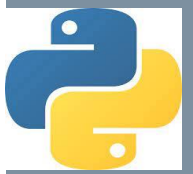
Nested If: You can have if statements inside if statements, this is called *nested* if statements.

```
x = 41
if x > 10:
  print("Above ten,")
  if x > 20:
    print("and also above 20!")
  else:
    print("but not above 20.")
```

Python Conditions

The pass Statement: if statements cannot be empty, but if you for some reason have an if statement with no content, put in the pass statement to avoid getting an error.

```
a = 33
b = 200

if b > a:
  pass

# having an empty if statement like this,
would raise an error without the pass
statement
```



Python LOOP

Python Loops: Python has two primitive loop commands:

- while loops
- for loops

The while Loop: With the while loop we can execute a set of statements as long as a condition is true.

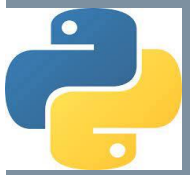
```
i = 1
while i < 6:
  print(i)
  i += 1
```

The break Statement: With the break statement we can stop the loop even if the while condition is true:

```
i = 1
while i < 6:
  print(i)
  if i == 3:
    break
  i += 1
```

The continue Statement: With the continue statement we can stop the current iteration, and continue with the next:

```
i = 0
while i < 6:
  i += 1
  if i == 3:
    continue
  print(i)
```



Python LOOP (FOR)

Python For Loops:

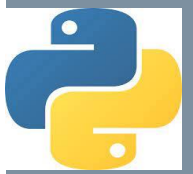
A for loop is used for iterating over a sequence (that is either a list, a tuple, a dictionary, a set, or a string).

This is less like the for keyword in other programming languages, and works more like an iterator method as found in other object-orientated programming languages.

With the for loop we can execute a set of statements, once for each item in a list, tuple, set etc.

Print each fruit in a fruit list:

```
fruits =
["apple", "banana", "cherry"]
for x in fruits:
  print(x)
```



Python LOOP (FOR)

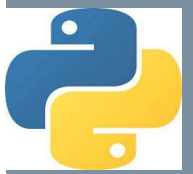
Looping Through a String: Even strings are iterable objects, they contain a sequence of characters:

```
for x in "banana":
  print(x)
```

The **break Statement:** With the break statement we can stop the loop before it has looped through all the items:

```
fruits =
["apple", "banana", "cherry"]
for x in fruits:
  if x == "banana":
    break
  print(x)
```

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
  print(x)
  if x == "banana":
    break
```



Python LOOP (FOR)

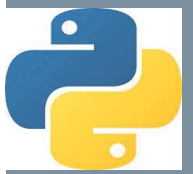
The continue Statement: With the continue statement we can stop the current iteration of the loop, and continue with the next:

```
fruits =
["apple", "banana",
"cherry"]
for x in fruits:
  if x == "banana":
    continue
  print(x)
```

The range() Function: To loop through a set of code a specified number of times, we can use the range() function,

The range() function returns a sequence of numbers, starting from 0 by default, and increments by 1 (by default), and ends at a specified number.

```
for x in range(6):
  print(x)
```



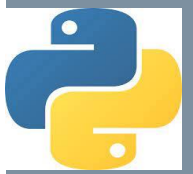
Python LOOP (FOR)

The range() function defaults to 0 as a starting value, however it is possible to specify the starting value by adding a parameter: range(2, 6), which means values from 2 to 6 (but not including 6):

```
for x in range(2, 6):
  print(x)
```

The range() function defaults to increment the sequence by 1, however it is possible to specify the increment value by adding a third parameter: range(2, 30, **3**):

```
for x in range(2, 30, 3):
  print(x)
```



Python LOOP (FOR)

Else in For Loop: The else keyword in a for loop specifies a block of code to be executed when the loop is finished:

```
for x in range(6):
  print(x)
else:
  print("Finally finished!")
```

Break the loop when x is 3, and see what happens with the else block:

```
for x in range(6):
  if x == 3: break
  print(x)
else:
  print("Finally finished!")
```

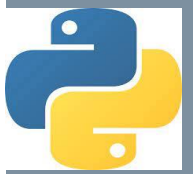
Python LOOP (FOR)

Nested Loops: A nested loop is a loop inside a loop.
The "inner loop" will be executed one time for each iteration of the "outer loop":

```
adj = ["red", "big", "tasty"]
fruits =
["apple", "banana", "cherry"]

for x in adj:
  for y in fruits:
    print(x, y)
```

The pass Statement: for loops cannot be empty, but if you for some reason have a for loop with no content, put in the pass statement to avoid getting an error.

```
for x in [0, 1, 2]:
  pass

# having an empty for loop like this, would raise an error without the pass statement
```



Python Functions

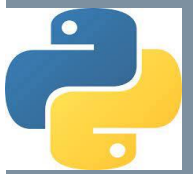
Python Functions: A function is a block of code which only runs when it is called. You can pass data, known as parameters, into a function. A function can return data as a result.

Creating a Function: In Python a function is defined using the def keyword:

```
def my_function():
  print("Hello from a function")
```

Calling a Function: To call a function, use the function name followed by parenthesis:

```
def my_function():
  print("Hello from a function")
my_function()
```



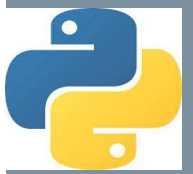
Python Functions

Arguments: Information can be passed into functions as arguments.

Arguments are specified after the function name, inside the parentheses. You can add as many arguments as you want, just separate them with a comma.

The following example has a function with one argument (fname). When the function is called, we pass along a first name, which is used inside the function to print the full name:

```
def my_function(fname):
  print(fname + " Refsnes")

my_function("Emil")
my_function("Tobias")
my_function("Linus")
```



Python Functions

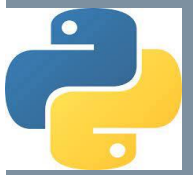
Parameters or Arguments?: The terms parameter and argument can be used for the same thing: information that are passed into a function.

From a function's perspective: A parameter is the variable listed inside the parentheses in the function definition.

An argument is the value that is sent to the function when it is called.

Number of Arguments: By default, a function must be called with the correct number of arguments. Meaning that if your function expects 2 arguments, you have to call the function with 2 arguments, not more, and not less.

```
def my_function(fname, lname):
  print(fname + " " + lname)

my_function("Emil", "Refsnes")
```



Python Functions

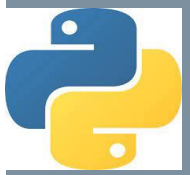
If you try to call the function with 1 or 3 arguments, you will get an error:

```
def my_function(fname, lname):
  print(fname + " " + lname)

my_function("Emil")
```

Arbitrary Arguments, *args: If you do not know how many arguments that will be passed into your function, add a * before the parameter name in the function definition.

This way the function will receive a *tuple* of arguments, and can access the items accordingly:

```
def my_function(*kids):
  print("The youngest child is " + kids[2])

my_function("Emil", "Tobias", "Linus")
```



Python Functions

Keyword Arguments: You can also send arguments with the *key = value* syntax. This way the order of the arguments does not matter.

```
def my_function(child3, child2, child1):
  print("The youngest child is " + child3)

my_function(child1 = "Emil", child2 = "Tobias",
child3 = "Linus")
```

Python Functions

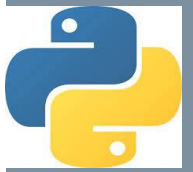
Arbitrary Keyword Arguments, **kwargs: If you do not know how many keyword arguments that will be passed into your function, add two asterisk: ** before the parameter name in the function definition.
This way the function will receive a *dictionary* of arguments, and can access the items accordingly:

```
def my_function(**kid):
  print("His last name is " + kid["lname"])

my_function(fname = "Tobias", lname = "Refsnes")
```

Python Functions

Default Parameter Value: The following example shows how to use a default parameter value. If we call the function without argument, it uses the default value:

```
def my_function(country = "Norway"):
  print("I am from " + country)

my_function("Sweden")
my_function("India")
my_function()
my_function("Brazil")
```



Python Functions

Passing a List as an Argument: You can send any data types of argument to a function (string, number, list, dictionary etc.), and it will be treated as the same data type inside the function.

E.g. if you send a List as an argument, it will still be a List when it reaches the function:

```
def my_function(food):
  for x in food:
    print(x)
fruits = ["apple", "banana", "cherry"]

my_function(fruits)
```

Python Functions

Passing a List as an Argument: You can send any data types of argument to a function (string, number, list, dictionary etc.), and it will be treated as the same data type inside the function.

E.g. if you send a List as an argument, it will still be a List when it reaches the function:

```
def my_function(food):
  for x in food:
    print(x)
fruits = ["apple", "banana", "cherry"]

my_function(fruits)
```

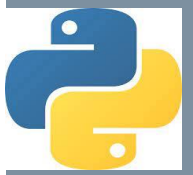
Python Functions

Return Values: To let a function return a value, use the return statement:

```
def my_function(x):
  return 5 * x

print(my_function(3))
print(my_function(5))
print(my_function(9))
```

The pass Statement: function definitions cannot be empty, but if you for some reason have a function definition with no content, put in the pass statement to avoid getting an error.

```
def myfunction():
  pass
```

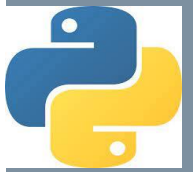


Python Functions

Recursion: Python also accepts function recursion, which means a defined function can call itself.

Recursion is a common mathematical and programming concept. It means that a function calls itself. This has the benefit of meaning that you can loop through data to reach a result.

The developer should be very careful with recursion as it can be quite easy to slip into writing a function which never terminates, or one that uses excess amounts of memory or processor power. However, when written correctly recursion can be a very efficient and mathematically-elegant approach to programming.

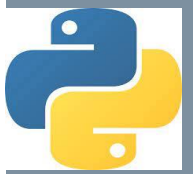


Python Functions

Recursion: In this example, tri_recursion() is a function that we have defined to call itself ("recurse"). We use the k variable as the data, which decrements (-1) every time we recurse. The recursion ends when the condition is not greater than 0 (i.e. when it is 0).

To a new developer it can take some time to work out how exactly this works, best way to find out is by testing and modifying it.

```
def tri_recursion(k):
  if(k > 0):
    result = k + tri_recursion(k - 1)
    print(result)
  else:
    result = 0
  return result

print("\n\nRecursion Example Results")
tri_recursion(6)
```



Python Lambda

Lambda Function: A lambda function is a small anonymous function.

A lambda function can take any number of arguments but can only have one expression.

Syntax:

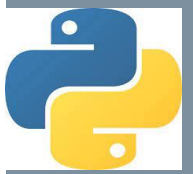
```
lambda arguments : expression
```

```
x = lambda a: a + 10
print(x(5))
```

Lambda functions can take any number of arguments:

```
x = lambda a, b : a * b
print(x(5, 6))
```

```
x = lambda a,b,c :a+ b + c
print(x(5, 6, 2))
```



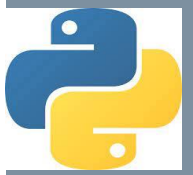
Python Lambda

The power of lambda is better shown when you use them as an anonymous function inside another function.
Say you have a function definition that takes one argument, and that argument will be multiplied with an unknown number:

```
def myfunc(n):
  return lambda a : a * n

mydoubler = myfunc(2)

print(mydoubler(11))
```

use the same function definition to make both functions, in the same program:

```
def myfunc(n):
  return lambda a : a * n

mydoubler = myfunc(2)
mytripler = myfunc(3)

print(mydoubler(11))
print(mytripler(11))
```




thank you